

*A RESEARCH PAPER ON*

***INSTA MECHANICS –ONLINE VEHICLE REPAIR & ASSISTANCE  
PLATFORM***

Submitted in the partial fulfillment of the Requirement for the Degree of

**BACHELOR OF TECHNOLOGY**

in

**COMPUTER SCIENCE & ENGINEERING**

by

**Kumar Raj Ranjan (23BTCSE0041)**

**Vihan Mudgal (23BTCSE 0060)**

**Umesh Roy (23BTCSE0283)**

Under the Supervision of

**Mr. Rakesh Arya**

Professor, Department of Computer Science and Engineering



Submitted to

Department Of Computer Science and Engineering

Dev Bhoomi Uttarakhand University, Uttarakhand -248001

## Abstract

Abstract — The **Insta Mechanic** project is designed as a web-based MVP platform that connects customers with verified mechanics in real time. The system enables customers to book services, mechanics to accept and complete jobs, and administrators to oversee verification and booking management. The frontend is built using **HTML5, CSS3, and Bootstrap 5**, ensuring responsive and mobile-first layouts, while the backend leverages **Node.js/Express** with **PostgreSQL** for secure data handling. The roadmap follows a phased approach, beginning with planning and design, progressing through frontend prototyping, backend API development, dashboards, real-time updates, payments, and deployment. Each phase includes acceptance criteria, testing strategies, and security measures. The MVP focuses on one city initially, with scalability planned for future expansion. By integrating payment gateways, geolocation, and real-time notifications, Insta Mechanic demonstrates how structured execution can transform an idea into a functional product within 12 weeks. This report provides a comprehensive analysis of the technical stack, architecture, challenges, and future enhancements, serving as both a case study and a practical guide for web application development. This review paper examines existing on-demand vehicle repair platforms and compares their operational models with Uber-like service ecosystems. It evaluates supporting technologies including client–server architecture, secure payment systems, scalability mechanisms, and predictive maintenance frameworks. The analysis highlights key strengths such as convenience and transparency, while identifying persistent gaps in fraud prevention, mechanic verification, rural accessibility, and predictive analytics. By synthesizing industry practices and academic research, the paper underscores the need for innovation in vehicle repair services.

Future directions are proposed, including AI-driven mechanic matching, microservices-based scalability, cloud-native expansion, and Zero Trust security models. The review concludes that while current platforms provide partial solutions, a comprehensive ecosystem integrating repair, towing, insurance, and preventive maintenance is essential. This study contributes to both academic and industrial discourse by offering a holistic perspective on how digital ecosystems can transform vehicle repair services into scalable, secure, and customer-centric platforms.

# Introduction

The increasing reliance on personal vehicles in urban areas has created a pressing demand for **on-demand repair and maintenance services**. Traditional roadside assistance often suffers from inefficiencies such as unverified mechanics, lack of transparency in pricing, and delays in service delivery. Customers face uncertainty when searching for reliable mechanics, while mechanics lack a structured platform to connect with potential clients.

The **Insta Mechanic** project addresses this challenge by creating a responsive web platform that connects customers with verified mechanics in real time. The system is scoped as a **Minimum Viable Product (MVP)**, focusing on one city to validate the concept before scaling to multiple regions. The primary objective is to enable a seamless end-to-end booking flow: a customer requests a mechanic, receives a cost estimate, completes payment, and obtains confirmation, while the mechanic accepts and updates job status until completion.

The project emphasizes **mobile-first design, secure authentication, and real-time updates** using Socket.io. Payments are integrated via Razorpay/Stripe, ensuring financial reliability. The backend architecture supports scalability through PostgreSQL and REST APIs, while the frontend ensures accessibility and responsiveness.

This research paper presents a **step-by-step roadmap**, analyzing each phase in detail — from planning and prototyping to deployment and feedback. It highlights technical decisions, challenges, and acceptance criteria, offering a structured methodology for building service-oriented web applications.

## *.1 Background of Vehicle Service Industry*

The vehicle service industry has traditionally operated through **local garages and roadside mechanics**, where customers rely on personal networks or word-of-mouth to find help. While this model has served communities for decades, it suffers from several limitations:

- **Unpredictability:** Customers often face uncertainty about mechanic availability, service quality, and pricing.
- **Delays:** Emergency breakdowns require immediate attention, but traditional garages may not provide on-demand support.
- **Trust Deficit:** Without standardized reviews or ratings, customers struggle to assess mechanic reliability.

In recent years, **digital transformation** has reshaped industries like transportation (Uber, Ola), food delivery (Zomato, Swiggy), and home services (UrbanClap). These platforms introduced **real-time booking, transparent pricing, and customer reviews**, creating trust and convenience. However, the **vehicle repair sector remains under-digitized**, leaving a gap for innovation.

The **Insta Mechanic project** aims to fill this gap by offering a **digital platform for on-demand vehicle repair services**, combining the strengths of traditional mechanics with modern technology.

## **.2 Problem Statement**

Despite the growth of digital platforms, the vehicle repair industry faces persistent challenges:

- **Customer Challenges:** Difficulty in locating nearby mechanics, lack of transparent pricing, and fear of fraud in payments.
- **Mechanic Challenges:** Limited visibility to customers, irregular income, and absence of digital tools to manage jobs.
- **Admin Challenges:** No centralized system to verify mechanics, monitor disputes, or ensure compliance.

The **research problem** can be framed as:

*“How can a digital platform ensure transparency, trust, and efficiency in on-demand vehicle repair services while balancing customer convenience, mechanic empowerment, and administrative oversight?”*

## **.3 Objectives of Insta Mechanic**

The project sets out clear objectives:

- **Primary Objectives:**
  - Develop a seamless booking system for customers.
  - Enable mechanics to accept jobs and update progress in real time.
  - Ensure secure, traceable payments through integrated gateways.
  - Provide administrators with tools for verification, monitoring, and dispute resolution.
- **Secondary Objectives:**
  - Build customer trust through ratings and reviews.
  - Expand services beyond repair (towing, car wash, insurance).
  - Integrate predictive maintenance using AI to prevent breakdowns.

## **.4 Scope of the Project**

The scope defines the boundaries of Insta Mechanic:

- **Functional Scope:** Covers booking, mechanic assignment, payment, reviews, and admin oversight.
- **Geographical Scope:** Initially focused on urban areas with high vehicle density, scalable to multiple cities.
- **Technical Scope:**

- Web-based platform with responsive frontend.
- Backend APIs for booking, payments, and notifications.
- PostgreSQL database for structured data storage.
- Integration with external services (Razorpay/Stripe, Google Maps, Nodemailer).
- **Limitations:**
  - Internet dependency for real-time updates.
  - Mechanic verification challenges in rural areas.
  - Initial scaling limited by infrastructure costs.

## *.5 Research Methodology*

The methodology outlines how the project is studied and developed:

- **Approach:**
  - Analytical study of existing platforms (Uber, UrbanClap).
  - Design and development of Insta Mechanic prototype.
  - Testing through simulated use cases (battery breakdown, towing request).
- **Data Collection:**
  - Literature review of service platforms.
  - Customer surveys to identify pain points.
  - Mechanic interviews to understand operational challenges.
- **Evaluation Metrics:**
  - Booking success rate.
  - Payment reliability.
  - Customer satisfaction (reviews).
  - Mechanic earnings growth.

## **Historical Background of Digital Vehicle Repair Platforms**

The concept of organized vehicle repair services has evolved significantly over the past century, shaped by technological progress, urbanization, and changing consumer expectations. In the **early automotive era (1900s–1950s)**, repair services were primarily provided by small, local garages or individual mechanics. These setups were informal, community-based, and relied heavily on personal trust rather than standardized processes. Customers often had long-term relationships with their local mechanic, but service quality and pricing varied widely, with little transparency or regulation.

By the **mid-20th century (1960s–1980s)**, the rise of mass automobile ownership and expansion of highways created demand for more structured repair services. This period saw the emergence of branded service centers linked to automobile manufacturers. These centers introduced standardized procedures, trained technicians, and quality assurance, but they were often expensive and limited to urban areas. Independent garages continued

to thrive, but the lack of transparency and inconsistent service quality remained a challenge.

The **late 20th century (1990s–2000s)** marked the beginning of digitization in automotive services. With the spread of the internet, customers gained access to online directories and aggregator websites that listed nearby garages and mechanics. While these platforms improved accessibility, they offered little beyond contact information and did not address deeper issues such as fraud prevention, service quality, or predictive maintenance.

The **early 21st century (2010s)** witnessed the rise of app-based platforms such as GoMechanic in India and YourMechanic in the USA. These platforms transformed the industry by introducing online booking, upfront pricing, digital invoices, and verified reviews. They emphasized convenience and transparency, aligning with the digital habits of modern consumers. Doorstep repair services further enhanced customer experience, while multi-service ecosystems like UrbanClap expanded reach by integrating car care into broader service portfolios.

In the **current era (2020s onwards)**, the industry is experimenting with Uber-inspired real-time allocation models, GPS tracking, and dynamic pricing. These innovations improve efficiency and transparency but remain limited in scope, as they do not fully integrate predictive analytics, fraud prevention, or ecosystem services such as towing and insurance.

Thus, the historical background reveals a clear trajectory: **from informal local garages → branded service centers → online directories → app-based booking platforms → doorstep services → multi-service ecosystems → real-time allocation models**. Each stage addressed specific customer needs but left gaps in transparency, scalability, predictive intelligence, and ecosystem integration. The next evolutionary step, represented by platforms like **Insta Mechanic**, aims to bridge these gaps by creating a holistic, AI-driven, secure, and customer-centric ecosystem that redefines the standards of vehicle repair services.

## Literature Review .

### *.1 Existing Solutions in On-Demand Vehicle Repair*

- Discuss platforms like **GoMechanic**, **UrbanClap (Urban Company)**, and **YourMechanic (USA)**.
- Highlight their features: booking apps, mechanic verification, service catalogs, and payment integration.
- Identify limitations:
  - Limited geographical coverage.
  - Lack of real-time tracking.

- Weak integration of predictive analytics.
- Compare these with **Insta Mechanic's proposed model**, which emphasizes transparency, scalability, and AI-driven enhancements.

## *.2 Comparative Study of Uber-like Service Models*

- Analyze **Uber, Ola, and Lyft** as case studies of successful on-demand platforms.
- Key features:
  - Real-time booking and tracking.
  - Dynamic pricing models.
  - Customer reviews and ratings.
- Discuss how these principles can be adapted to vehicle repair services.
- Example: Uber's surge pricing → Insta Mechanic's surge pricing for peak service hours.

## *.3 Gaps in Current Systems*

- **Customer Perspective:**
  - Lack of transparency in pricing.
  - Difficulty in finding trusted mechanics.
- **Mechanic Perspective:**
  - Limited visibility to customers.
  - No structured earnings dashboard.
- **Admin Perspective:**
  - Weak dispute resolution mechanisms.
  - No centralized monitoring of fraud or compliance.
- Insta Mechanic addresses these gaps with:
  - Transparent pricing algorithms.
  - Real-time dashboards for mechanics.
  - Admin panel with dispute resolution and refund workflows.

## *.4 Academic References on Client-Server Models*

- Review scholarly work on **client-server architecture** in distributed systems.
- Discuss how separation of concerns (frontend vs backend) improves scalability.
- Cite studies on RESTful APIs, database normalization, and real-time communication protocols (WebSockets, Socket.io).
- Relate these to Insta Mechanic's design:
  - Client handles presentation.
  - Server manages business logic.
  - Database ensures persistence.

## *.5 Payment Security in Digital Platforms*

- Review literature on **secure payment systems** (PCI DSS compliance, encryption, fraud prevention).

- Discuss how platforms like PayPal, Razorpay, and Stripe ensure transaction integrity.
- Highlight mechanisms:
  - SSL/TLS encryption.
  - Webhook signature verification.
  - Audit logs for compliance.
- Show how Insta Mechanic integrates these practices to build customer trust.

## *.6 Predictive Maintenance & AI in Automotive Services*

- Review research on **predictive analytics in vehicle maintenance**.
- Discuss machine learning models that predict failures based on mileage, usage, and service history.
- Cite studies on AI-driven preventive maintenance in fleet management.
- Relate to Insta Mechanic's future scope:
  - AI-based mechanic matching.
  - Predictive alerts for customers.

# Autonomy and New Technologies in Digital Vehicle Repair Platforms

The next generation of platforms like **Insta Mechanic** is defined by two critical aspects: **autonomy** and the adoption of **new technologies**. Together, these elements transform traditional repair services into intelligent, self-regulating ecosystems.

## *Autonomy in Platforms*

Autonomy refers to the ability of the platform to operate with minimal human intervention while ensuring efficiency, transparency, and trust.

- **Automated Service Allocation:** AI-driven algorithms autonomously assign mechanics to customers based on proximity, skill set, and workload, reducing delays and ensuring fairness.
- **Self-regulating Fraud Detection:** The system continuously monitors transactions, reviews, and mechanic activities to detect anomalies without requiring manual oversight.
- **Dynamic Pricing Models:** Prices are adjusted automatically based on demand, location, and service type, ensuring competitive yet fair rates.
- **Predictive Alerts:** Vehicles connected to the platform can autonomously send signals about potential issues (e.g., brake wear, oil degradation), triggering preventive service requests.
- **Zero Trust Security:** Every request is verified continuously, ensuring that no user or device is inherently trusted, thereby reducing risks of insider threats.



This autonomy reduces dependency on administrators, empowers mechanics, and enhances customer confidence by ensuring that the system itself enforces fairness and reliability.

### *New Technologies Driving Innovation*

The evolution of these platforms is powered by cutting-edge technologies:

- **Artificial Intelligence (AI) & Machine Learning**
  - Predictive maintenance through data analytics.
  - Intelligent mechanic matching based on expertise and customer needs.
  - Sentiment analysis of customer feedback for service improvement.
- **Blockchain Technology**
  - Tamper-proof service records and transaction logs.
  - Transparent mechanic certifications and customer reviews.
  - Secure insurance claim processing.
- **Cloud-Native Microservices**
  - Modular architecture for scalability and resilience.
  - Independent scaling of booking, payment, and review modules.
  - Faster deployment of new features with container orchestration (e.g., Kubernetes).
- **IoT Integration**
  - Real-time vehicle diagnostics through sensors.
  - Automated alerts for preventive maintenance.
  - Seamless communication between vehicles, mechanics, and customers.
- **Cybersecurity Frameworks**
  - Multi-factor authentication (MFA) for secure access.
  - End-to-end encryption for data protection.

## **Methodology**

The methodology adopted in this study is comprehensive, combining **exploratory research, comparative analysis, gap identification, technology mapping, and conceptual system design**. It ensures that the proposed Insta Mechanic framework is not only academically rigorous but also practically relevant. By systematically analyzing existing platforms, identifying research gaps, and proposing innovative solutions, the methodology bridges the divide between theory and practice.

This structured approach positions the research as a meaningful contribution to both academic discourse and industry innovation, paving the way for future empirical studies and real-world implementation.

### *1. Research Design*

The study follows a **qualitative, exploratory, and comparative design**.

- **Qualitative:** Since the focus is on understanding processes, technologies, and customer experiences rather than numerical data, qualitative methods are most suitable.
- **Exploratory:** Digital vehicle repair platforms are relatively new, and their integration with advanced technologies like AI, blockchain, and IoT is still evolving. Hence, exploratory research helps uncover emerging trends and possibilities.
- **Comparative:** By analyzing platforms such as GoMechanic, YourMechanic, UrbanClap, and Uber-inspired models, the study highlights strengths, weaknesses, and gaps.
- **Conceptual:** The final stage involves proposing a new framework—Insta Mechanic—that integrates innovative technologies to bridge identified gaps.

This design ensures that the research is not only descriptive but also forward-looking, offering actionable insights for both academia and industry.

## ***2. Data Collection***

Data collection was conducted through **secondary sources** and **conceptual primary insights**.

- **Secondary Sources:**
  - Academic journals on predictive maintenance, cyber-physical systems, and AI applications in service industries.
  - Industry reports from McKinsey, Deloitte, PwC, and Frost & Sullivan on automotive aftersales and digital transformation.
  - Case studies of existing platforms to understand operational models, customer feedback, and scalability challenges.
- **Primary Insights (Conceptual):**
  - Brainstorming sessions among the project team (Kumar Raj Ranjan, Vihan Moudgil, and Umesh Roy).
  - Technical discussions on feasibility of integrating AI-driven mechanic matching, blockchain-based fraud prevention, and microservices scalability.
  - Collaborative evaluation of customer pain points and mechanic challenges to ensure practical relevance.

This dual approach ensured that the research was grounded in both **academic theory** and **real-world industry practices**.

## ***3. Analytical Framework***

The analytical framework combined multiple methods to ensure depth and clarity.

- **Gap Analysis:** Identified shortcomings in current platforms, such as lack of predictive maintenance, weak fraud prevention, limited scalability, and poor ecosystem integration.
- **SWOT Analysis:** Evaluated strengths (convenience, transparency), weaknesses (urban-centric, reactive models), opportunities (AI, blockchain, IoT), and threats (cybersecurity risks, competition).
- **Technology Mapping:** Aligned emerging technologies with identified gaps. For example, AI was mapped to predictive maintenance, blockchain to fraud prevention, and cloud-native microservices to scalability.

This structured analysis provided a clear roadmap for designing the proposed solution.

#### *4. Proposed System Development*

The methodology included designing a **conceptual system architecture** for Insta Mechanic.

- **Architecture Design:**
  - **Security Layer:** Multi-factor authentication, role-based access control, encryption, and Zero Trust security.
  - **Service Layer:** AI modules for predictive maintenance, mechanic matching, and fraud detection.
  - **Data Layer:** Blockchain ledger for tamper-proof records and transparent transactions.
  - **Infrastructure Layer:** Cloud-native microservices for scalability, resilience, and modular deployment.
- **Ecosystem Integration:**
  - Unified platform offering repair, towing, insurance claim processing, and preventive maintenance.
  - Dashboards for customers (booking, tracking, invoices), mechanics (job allocation, certification), and administrators (fraud monitoring, compliance).

This design ensures that the proposed system is **secure, scalable, intelligent, and customer-centric**.

#### *5. Validation Approach*

Since the project is conceptual, validation was conducted through **comparative benchmarking** and **alignment with research gaps**.

- **Comparative Benchmarking:** The proposed features were compared against existing platforms to demonstrate improvements. For example, while GoMechanic offers standardized pricing, Insta Mechanic adds predictive analytics and fraud prevention.

- **Alignment with Research Gaps:** Each proposed innovation was mapped to a specific gap identified in literature and practice. For instance, predictive maintenance addressed the gap of reactive models, while blockchain addressed transparency issues.
- **Future Scope Analysis:** The framework was evaluated for adaptability to emerging technologies such as 5G connectivity, digital twins, and generative AI.

This validation ensured that the proposed solution was academically sound and practically feasible.

## 6. Limitations

The methodology acknowledges certain limitations:

- The study is primarily conceptual and based on secondary data; practical implementation and testing remain future work.
- Regional variations (urban vs. rural adoption) may affect scalability and accessibility.
- Customer behavior and mechanic acceptance of new technologies may vary, requiring further empirical research.

By recognizing these limitations, the study maintains transparency and sets the stage for future empirical validation.

## 7. Ethical Considerations

Ethical integrity was maintained throughout the research:

- Reliance on credible academic and industry sources ensured accuracy.
- Transparency in acknowledging limitations avoided overstating findings.
- The proposed system emphasized customer privacy and data protection, aligning with GDPR and PCI DSS standards

# System Design

The **system design of Insta Mechanic** is the backbone of the entire project, as it defines how different components interact, how data flows between them, and how the platform ensures reliability, scalability, and security. At its core, the system is built around three primary personas: the **customer**, the **mechanic**, and the **administrator**. Each persona has distinct responsibilities and interfaces, but they are interconnected through a unified backend and database.

For the **customer**, the system provides a responsive web interface where they can request services by selecting issue types, entering location details, and confirming bookings with payment. The frontend is designed using **HTML5, CSS3, and Bootstrap 5**, ensuring mobile-first responsiveness and accessibility. Customers interact with forms and widgets that validate inputs, calculate cost estimates, and display booking confirmations.

The **mechanic** interacts with the system through a dedicated dashboard. This dashboard allows mechanics to toggle availability, view assigned jobs, accept or decline bookings, and update job statuses in real time. The backend ensures that mechanics only see jobs relevant to their location and service offerings, using geolocation queries powered by the **Haversine formula**. Socket.io enables real-time updates, so mechanics and customers are instantly notified of status changes.

The **administrator** oversees the platform through an admin panel. This panel provides visibility into all bookings, mechanics, and payments. Admins can verify mechanics through KYC checks, override booking statuses in case of disputes, and initiate refunds. The admin panel is critical for maintaining trust and accountability, ensuring that only verified mechanics are visible to customers and that disputes are resolved fairly.

From a technical perspective, the system follows a **layered architecture**. The frontend communicates with the backend via RESTful APIs built on **Node.js and Express**. The backend enforces authentication using **JWT tokens**, ensuring secure role-based access control. The database layer is managed by **PostgreSQL**, chosen for its relational consistency and ACID compliance. Tables are designed to capture users, mechanics, services, bookings, and payments, with relationships defined through foreign keys. This schema ensures data integrity and supports complex queries, such as filtering bookings by status or calculating mechanic earnings.

The system also integrates with external services. Payment gateways such as **Razorpay or Stripe** handle financial transactions, with webhook verification ensuring authenticity. Email notifications are sent via **Nodemailer**, while SMS alerts can be added through services like Twilio. Hosting is distributed across **Netlify/Vercel** for the frontend and **Render/Railway** for the backend, with PostgreSQL hosted on Railway or Supabase. Monitoring tools such as UptimeRobot and Google Analytics track system performance and user activity.

Security is embedded throughout the design. Passwords are hashed using bcrypt, JWT tokens have short expiry times, and rate limiting prevents brute-force attacks. Helmet middleware adds secure HTTP headers, while CORS restricts API access to trusted domains. Error handling is standardized, with JSON responses providing clear messages without exposing sensitive details.

In summary, the system design of Insta Mechanic balances **user-centric functionality** with **technical robustness**. It ensures that customers can book services easily, mechanics can manage jobs efficiently, and administrators can maintain oversight. By combining responsive frontend design, scalable backend architecture, secure database management,

and real-time communication, the system creates a reliable and transparent platform for on-demand vehicle repair services.

## System Architecture

The **system architecture of Insta Mechanic** is designed as a layered, modular framework that ensures scalability, maintainability, and secure communication between components. At the highest level, the architecture is divided into three layers: the **presentation layer (frontend)**, the **application layer (backend APIs)**, and the **data layer (database and external services)**. Each layer has distinct responsibilities, and together they form a cohesive ecosystem that supports customers, mechanics, and administrators.

The **presentation layer** consists of the responsive web interface built with **HTML5, CSS3, and Bootstrap 5**. This layer is responsible for capturing user input, displaying booking forms, dashboards, and admin panels, and rendering real-time updates. It communicates exclusively with the backend through RESTful APIs, ensuring that the frontend remains lightweight and focused on user experience. The frontend also integrates with external services such as Google Maps for location visualization and Razorpay/Stripe for payment checkout modals.

The **application layer** is powered by **Node.js and Express**, serving as the central hub for business logic. It exposes RESTful endpoints for authentication, booking management, mechanic availability, and admin oversight. Middleware enforces **JWT-based authentication** and role-based access control, ensuring that only authorized users can access specific routes. Real-time communication is handled by **Socket.io**, which establishes persistent connections between clients and the server, enabling instant updates for booking status and mechanic location. The backend also integrates with external services such as payment gateways and email/SMS providers, using secure APIs and webhook verification to maintain trust and reliability.

The **data layer** is managed by **PostgreSQL**, chosen for its relational consistency and ability to handle complex queries. The schema is designed with normalized tables for users, mechanics, services, bookings, and payments, with foreign keys maintaining relationships between entities. This ensures data integrity and supports advanced queries, such as filtering bookings by status or calculating mechanic earnings. ORM tools like Prisma or Sequelize simplify migrations and queries, making the database layer easier to maintain and extend.

External services form an important part of the architecture. Payment gateways such as Razorpay or Stripe handle financial transactions, with webhook verification ensuring authenticity. Email notifications are sent via Nodemailer, while SMS alerts can be integrated through Twilio. Hosting is distributed across **Netlify/Vercel** for the frontend

and **Render/Railway** for the backend, with PostgreSQL hosted on Railway or Supabase. Monitoring tools such as UptimeRobot and Google Analytics provide visibility into system performance and user activity, while error tracking tools like Sentry capture exceptions for debugging.

Security is embedded across all layers. The frontend enforces input validation, the backend hashes passwords with bcrypt and issues short-lived JWT tokens, and the database is protected with role-based access and SSL connections. Middleware such as Helmet adds secure HTTP headers, while CORS restricts API access to trusted domains. Rate limiting prevents brute-force login attempts, and audit logs track admin actions for compliance.

In summary, the system architecture of Insta Mechanic is a **layered, service-oriented design** that balances simplicity with scalability. The frontend provides a user-friendly interface, the backend enforces business logic and security, and the database ensures reliable persistence. External services extend functionality for payments, notifications, and monitoring. Together, these components create a robust platform capable of supporting real-time, on-demand vehicle repair services while remaining flexible enough to evolve into a larger ecosystem

## Client–Server Architecture

The **client–server architecture** of the Insta Mechanic project is designed to separate responsibilities between the user-facing interface (client) and the backend logic (server), ensuring scalability, security, and efficient communication. This model follows the classic principle where the client initiates requests and the server processes them, returning structured responses.

On the **client side**, customers, mechanics, and administrators interact with the system through responsive web pages built using **HTML5, CSS3, and Bootstrap 5**. The client is responsible for capturing user input, validating it locally, and sending it to the server via RESTful API calls. For example, when a customer submits a booking form, the client packages the data into a JSON request and sends it to the backend. Similarly, mechanics use their dashboard to update job statuses, and admins use their panel to verify mechanics or manage disputes. The client is lightweight, focusing primarily on presentation and user experience, while delegating all business logic to the server.

On the **server side**, the backend is implemented using **Node.js and Express**, which acts as the central hub for processing requests. The server receives API calls from the client, authenticates them using **JWT tokens**, and executes the appropriate business logic. Controllers handle specific tasks such as booking creation, mechanic availability updates,

and payment verification. Middleware enforces security policies, including input validation, role-based access control, and rate limiting. The server communicates with the **PostgreSQL database**, ensuring that all data — users, bookings, services, payments, and reviews — is stored consistently and securely.

The **database layer** plays a critical role in the client–server model. PostgreSQL maintains relational integrity by linking customers, mechanics, services, and payments through foreign keys. When the server processes a request, it queries or updates the database and returns the results to the client. For example, when a mechanic accepts a booking, the server updates the booking record in the database and sends a confirmation back to the client.

The architecture also integrates **external services** into the client–server flow. Payment gateways such as Razorpay or Stripe handle financial transactions, with the server acting as the intermediary. The client initiates payment, the server creates an order, and the gateway processes the transaction. Webhooks from the gateway notify the server of payment success or failure, which is then reflected in the database and communicated back to the client. Similarly, email notifications are sent via Nodemailer, and real-time updates are managed through **Socket.io**, which maintains persistent connections between clients and the server.

Security is embedded throughout the client–server architecture. The client enforces input validation, while the server hashes passwords with bcrypt, issues short-lived JWT tokens, and verifies payment signatures. CORS policies restrict API access to trusted domains, and Helmet middleware adds secure HTTP headers. This layered security ensures that sensitive data remains protected during transmission and storage.

## Deployment Architecture

The **deployment architecture** of Insta Mechanic describes how the system’s components are hosted, connected, and maintained in a production environment. It ensures that the platform is scalable, reliable, and secure while delivering real-time services to customers and mechanics.

### *1. Frontend Deployment*

- The **frontend** (customer booking interface, mechanic dashboard, admin panel) is built with **HTML5, CSS3, Bootstrap, and JavaScript**.
- It is hosted on **Netlify or Vercel**, which provide:
  - Global CDN distribution for fast page loads.
  - Automatic SSL certificates for secure HTTPS connections.
  - Continuous deployment pipelines linked to GitHub/GitLab repositories.
- Static assets (CSS, JS, images) are cached at edge servers, ensuring low latency for users across multiple regions.



## *2. Backend Deployment*

- The **backend** is powered by **Node.js and Express**, hosted on **Render, Railway, or Heroku**.
- Responsibilities:
  - Handle API requests (authentication, booking, payments, reviews).
  - Manage real-time communication via **Socket.io**.
  - Integrate with external services (payment gateways, email/SMS).
- Deployment features:
  - Auto-scaling containers to handle traffic spikes.
  - Load balancing to distribute requests across multiple instances.
  - Environment variables stored securely for API keys and database credentials.

## *3. Database Deployment*

- The **PostgreSQL database** is hosted on **Railway, Supabase, or AWS RDS**.
- Features:
  - ACID compliance for reliable transactions.
  - SSL encryption for secure connections.
  - Automated backups and point-in-time recovery.
  - Read replicas for scaling queries under heavy load.
- Schema includes tables for Users, Mechanics, Services, Bookings, Payments, Reviews, and Vehicles.

## *4. External Services Integration*

- **Payment Gateways:** Razorpay/Stripe handle transactions. Webhooks notify backend of payment status.
- **Email Notifications:** Nodemailer sends booking confirmations and receipts.
- **SMS Alerts:** Twilio (optional) for urgent updates.
- **Maps Integration:** Google Maps API provides location visualization and ETA tracking.

## *5. Monitoring & Logging*

- **UptimeRobot** monitors backend uptime.
- **Google Analytics** tracks frontend activity.
- **Sentry** captures backend errors and exceptions.
- **Audit Logs** record admin actions for compliance.

## *6. Security Deployment*

- **HTTPS enforced** across frontend and backend.
- **JWT authentication** for secure API access.
- **bcrypt password hashing** for user credentials.

- **Helmet middleware** adds secure HTTP headers.
- **CORS policies** restrict API access to trusted domains.
- **Rate limiting** prevents brute-force login attempts.

### Text-Based Deployment Diagram

#### Code

```
Frontend (Netlify/Vercel) → HTTPS → Backend (Node.js/Express on  
Render/Railway)  
Backend ↔ PostgreSQL Database (Railway/Supabase/AWS RDS)  
Backend ↔ Payment Gateway (Razorpay/Stripe Webhooks)  
Backend ↔ Email/SMS Services (Nodemailer/Twilio)  
Backend ↔ Maps API (Google Maps)  
Monitoring Tools (UptimeRobot, Sentry, Google Analytics) → Track System  
Health
```

## Scalability & Performance Architecture

The **scalability and performance architecture** of Insta Mechanic ensures that the system can handle high traffic, multiple concurrent bookings, and simultaneous payments without downtime. It is designed to grow horizontally and vertically, balancing loads across components while maintaining responsiveness.

### 1. Horizontal Scaling

- **Frontend:** Hosted on Netlify/Vercel with CDN distribution. Static assets are cached at edge servers, so thousands of customers can access booking forms simultaneously without bottlenecks.
- **Backend:** Node.js/Express deployed on Render/Railway with auto-scaling containers. When traffic spikes (e.g., during peak hours), new instances are spun up automatically.
- **Database:** PostgreSQL supports read replicas, allowing queries to be distributed across multiple servers. Heavy read operations (like booking searches) are offloaded to replicas, while write operations remain on the primary node.

## 2. Load Balancing

- Requests are distributed across backend instances using **load balancers**.
- This prevents any single server from being overwhelmed and ensures consistent response times.
- Sticky sessions are used for real-time Socket.io connections, ensuring customers and mechanics maintain uninterrupted communication.

## 3. Caching Layer

- **Redis or Memcached** is used to cache frequently accessed data, such as service categories, mechanic availability, and booking status.
- This reduces database load and speeds up response times.
- Example: When a customer searches for “Battery Replacement,” cached results are returned instantly instead of querying the database repeatedly.

## 4. Asynchronous Processing

- Background jobs (e.g., sending emails, SMS alerts, analytics logging) are handled by **message queues** like RabbitMQ or Bull (Redis-based).
- This ensures that critical booking and payment flows remain fast, while non-critical tasks are processed asynchronously.

## 5. Database Optimization

- Tables are normalized to avoid redundancy.
- Indexes are applied on frequently queried fields (e.g., booking status, mechanic location).
- Partitioning is used for large tables like `Bookings` and `Payments` to improve query performance.

## 6. Payment Scalability

- Payment gateways (Razorpay/Stripe) are designed to handle thousands of transactions per second.
- Webhooks are processed asynchronously to avoid blocking booking flows.
- Failed payments are retried automatically, and audit logs ensure traceability.

## 7. Monitoring & Auto-Healing

- **UptimeRobot** and **Prometheus/Grafana** monitor system health.
- If an instance fails, orchestration tools like **Docker Swarm/Kubernetes** restart containers automatically.
- Alerts are sent to administrators for anomalies (e.g., sudden surge in failed payments).

## 8. Security at Scale

- Rate limiting prevents denial-of-service attacks.
- JWT tokens expire quickly to reduce risk of replay attacks.
- Database connections are encrypted with SSL, even under high load.

### Text-Based Scalability Flow Diagram

Frontend (Netlify/Vercel CDN) → Load Balancer → Backend Instances (Node.js/Express)

Backend ↔ Redis Cache (Fast Data Access)

Backend ↔ PostgreSQL (Primary + Read Replicas)

Backend ↔ Message Queue (Async Jobs: Emails, SMS, Analytics)

Backend ↔ Payment Gateway (Razorpay/Stripe Webhooks)

Monitoring Tools (Prometheus, UptimeRobot, Grafana) → Auto-Healing & Alerts

# Future Enhancements Architecture

The **future enhancements architecture** of Insta Mechanic envisions how the platform can evolve beyond its current deployment, scalability, and performance design. These enhancements focus on integrating advanced technologies, expanding services, and restructuring components into microservices for greater flexibility.

## 1. AI-Based Mechanic Matching

- **Enhancement:** Introduce machine learning models to intelligently match customers with mechanics.
- **How it works:**
  - AI analyzes historical booking data, mechanic ratings, location, and availability.
  - Predictive algorithms suggest the “best-fit” mechanic for each request.
- **Architecture Impact:**
  - A new **AI Recommendation Service** microservice is added.
  - It communicates with the backend via APIs and queries the database for training data.

## 2. Predictive Maintenance Services

- **Enhancement:** Offer proactive alerts to customers about potential vehicle issues before breakdowns occur.
- **How it works:**
  - Customers input vehicle details (make, model, mileage).
  - AI models predict common failures (battery, tires, oil changes).
  - Notifications are sent recommending preventive service bookings.
- **Architecture Impact:**
  - A **Predictive Analytics Engine** microservice is integrated.
  - It uses historical service data and external automotive datasets.

## 3. Microservices Architecture

- **Enhancement:** Transition from monolithic backend to microservices.
- **How it works:**
  - Separate services for authentication, booking, payments, notifications, and reviews.
  - Each service runs independently in containers (Docker/Kubernetes).
- **Architecture Impact:**
  - Improves scalability and fault isolation.
  - Allows independent deployment and scaling of high-demand services (e.g., payments).

#### 4. Cloud-Native Expansion

- **Enhancement:** Deploy on **AWS/GCP/Azure** for global reach.
- **How it works:**
  - Use managed services like AWS RDS (PostgreSQL), Lambda (serverless functions), and S3 (storage).
  - Auto-scaling groups handle traffic surges.
- **Architecture Impact:**
  - Improves resilience with multi-region deployments.
  - Enables disaster recovery and high availability.

#### 5. Advanced Security & Compliance

- **Enhancement:** Implement **Zero Trust Architecture** and compliance with GDPR/PCI DSS.
- **How it works:**
  - Continuous authentication for every API call.
  - Data encryption at rest and in transit.
  - Role-based access with fine-grained policies.
- **Architecture Impact:**
  - Adds a **Security Gateway Service** for centralized policy enforcement.

#### 6. Extended Ecosystem Services

- **Enhancement:** Expand beyond mechanics to include towing, car wash, insurance, and roadside assistance.
- **How it works:**
  - Modular service catalog in the database.
  - Customers can book multiple services in one request.
- **Architecture Impact:**
  - Requires **Service Aggregator Microservice** to manage multi-service bookings.

#### Text-Based Future Architecture Diagram

Frontend (Web/Mobile Apps) → API Gateway → Microservices:

- Auth Service
- Booking Service
- Payment Service
- Notification Service
- Review Service
- AI Recommendation Service
- Predictive Analytics Engine
- Service Aggregator

Microservices ↔ PostgreSQL (Primary + Replicas)

Microservices ↔ External Services (Payment Gateway, Email/SMS, Maps)

Microservices ↔ Cloud Services (AWS/GCP/Azure)

Monitoring & Security Gateway → Zero Trust + Compliance

# Complete System Architecture Blueprint

Now that we've explored each layer separately — client-server, deployment, scalability, and future enhancements — let's combine them into a **single unified architecture blueprint**. This blueprint shows how all components interact, evolve, and scale together, forming the backbone of the **Insta Mechanic ecosystem**.

## 1. Client Layer (User Interfaces)

- **Customers:** Booking form, payment interface, service tracking.
- **Mechanics:** Dashboard for availability, job acceptance, progress updates, earnings.
- **Admins:** Admin panel for verification, monitoring, dispute resolution.
- **Technology:** HTML5, CSS3, Bootstrap, JavaScript → hosted on Netlify/Vercel with CDN caching.

## 2. Application Layer (Backend Services)

- **Core Services:** Authentication, Booking, Payment, Notification, Review.
- **Real-Time Communication:** Socket.io for live booking updates and mechanic tracking.
- **Business Logic Controllers:**
  - `AuthController` → login/register.
  - `BookingController` → create/update bookings.
  - `MechanicController` → availability & job progress.
  - `PaymentController` → order creation, webhook verification.
  - `AdminController` → verification, refunds, overrides.
- **Technology:** Node.js + Express → deployed on Render/Railway/Heroku with auto-scaling containers.

## 3. Data Layer (Database)

- **PostgreSQL** hosted on Railway/Supabase/AWS RDS.
- **Tables:** Users, Mechanics, Services, Bookings, Payments, Reviews, Vehicles.
- **Features:**
  - ACID compliance for transaction reliability.
  - Read replicas for scalability.
  - Indexing & partitioning for performance.
  - Automated backups for disaster recovery.

## 4. External Services Integration

- **Payment Gateways:** Razorpay/Stripe for secure transactions.
- **Email/SMS:** Nodemailer, Twilio for notifications.
- **Maps API:** Google Maps for location tracking & ETA.

- **Monitoring Tools:** UptimeRobot, Sentry, Google Analytics.

### 5. Security Layer

- **Frontend:** HTTPS enforced, client-side validation.
- **Backend:** JWT authentication, bcrypt password hashing, Helmet middleware, CORS restrictions.
- **Database:** SSL encryption, role-based access.
- **Fraud Prevention:** Rate limiting, webhook signature verification, audit logs.

### 6. Scalability & Performance Layer

- **Load Balancing:** Distributes requests across backend instances.
- **Caching:** Redis/Memcached for frequently accessed data.
- **Async Processing:** RabbitMQ/Bull queues for background jobs (emails, analytics).
- **Auto-Healing:** Kubernetes/Docker orchestration restarts failed containers.
- **Monitoring:** Prometheus/Grafana dashboards for real-time performance metrics.

### 7. Future Enhancements Layer

- **AI Recommendation Service:** Smart mechanic matching.
- **Predictive Analytics Engine:** Preventive maintenance alerts.
- **Microservices Transition:** Independent services for auth, booking, payments, notifications.
- **Cloud-Native Expansion:** AWS/GCP/Azure multi-region deployments.
- **Extended Ecosystem:** Towing, car wash, insurance, roadside assistance.
- **Zero Trust Security Gateway:** Continuous authentication & compliance.

## Data Flow Explanation

The **data flow in Insta Mechanic** describes how information moves seamlessly between customers, mechanics, administrators, and the system's backend. This flow ensures that every action — from booking a service to completing a payment — is captured, processed, and communicated in real time.

The process begins with the **customer input stage**. When a customer visits the booking page, they enter details such as their name, contact information, vehicle type, issue description, and preferred time. The frontend validates this data using client-side checks before sending it to the backend via a secure REST API call. At this point, the booking request is transmitted to the **application layer**, where the backend processes it. The backend assigns a unique booking ID, calculates the estimated cost using predefined formulas, and stores the request in the **Bookings table** of the PostgreSQL database.

Once the booking is stored, the backend triggers **real-time communication** using Socket.io. A notification is sent to available mechanics in the system, informing them of



the new job request. The mechanic dashboard displays the booking details, including customer location and service type. When a mechanic accepts the job, the backend updates the booking status from “requested” to “accepted” and locks the record to prevent multiple mechanics from claiming the same job. This update is instantly broadcast back to the customer’s frontend, where they see confirmation that a mechanic is on the way.

As the mechanic progresses through the job, they update the status to “on\_the\_way,” “in\_progress,” and finally “completed.” Each status change flows from the mechanic’s dashboard to the backend, which validates the transition and updates the database. Socket.io then pushes these updates to the customer’s interface, ensuring they are informed in real time. Simultaneously, the system captures the mechanic’s location using periodic geolocation updates, storing coordinates in the database and embedding a Google Maps link for the customer to track ETA.

The **payment flow** begins once the job is completed. The customer initiates payment through Razorpay or Stripe, which generates an order ID linked to the booking. The payment gateway processes the transaction and sends a webhook to the backend, verifying authenticity. The backend updates the **Payments table**, marking the booking as “paid.” A receipt is generated, stored in the database, and emailed to the customer. The mechanic’s earnings dashboard is updated accordingly, showing the new income entry.

The **administrator’s role** in the data flow is oversight and intervention. Admins can view all bookings and payments through the admin panel, which queries the database for records. If disputes arise, admins can override booking statuses or initiate refunds. Refund requests flow from the admin panel to the backend, which communicates with the payment gateway to reverse the transaction. The database is updated to reflect the refund, and the customer is notified via email.

Throughout this flow, **security measures** ensure data integrity. JWT tokens authenticate each request, bcrypt secures passwords, and role-based access control prevents unauthorized actions. Error handling ensures that invalid inputs or failed transactions return clear JSON responses, while audit logs track admin interventions for compliance.

To make the **system design and data flow** more tangible, let’s walk through real-world use case scenarios that demonstrate how the Insta Mechanic platform operates end-to-end. These scenarios highlight the interactions between customers, mechanics, administrators, and the backend system, showing how data moves through the architecture in practical situations.

### *Scenario 1: Battery Breakdown on the Road*

Imagine a customer driving late at night when their car battery suddenly fails. The customer opens the **Insta Mechanic website** on their mobile phone and navigates to the booking form. They enter their details — name, phone number, vehicle type, and select

“Battery Replacement” as the issue. The system captures their location using the HTML5 Geolocation API and calculates an estimated cost based on the base fee plus distance.

This booking request flows to the backend, where the **BookingController** creates a new record in the `Bookings` table. A unique booking ID is generated, and the status is set to “requested.” Socket.io broadcasts this request to all available mechanics within the service radius. On the mechanic’s dashboard, the job appears with details of the customer’s location and issue.

One mechanic accepts the job, and the backend updates the booking status to “accepted.” This change is pushed back to the customer’s frontend, where they see confirmation that a mechanic is on the way. The mechanic’s location is periodically updated and shared via a Google Maps link, allowing the customer to track ETA. Once the mechanic arrives and replaces the battery, they update the status to “completed.” The backend verifies the transition, updates the database, and triggers a payment request.

The customer pays securely through Razorpay. The payment gateway processes the transaction and sends a webhook to the backend, which verifies authenticity and updates the `Payments` table. A receipt is generated, emailed to the customer, and stored in the database. The mechanic’s earnings dashboard reflects the new income entry, closing the loop of this scenario.

### *Scenario 2: Towing Request After Accident*

A customer experiences a minor accident and needs towing services. They use the booking form to request “Towing Assistance.” The backend processes the request, stores it in the database, and broadcasts it to nearby mechanics with towing capability.

A mechanic with a tow truck accepts the job. The booking status changes to “accepted,” and the customer receives real-time updates. The mechanic updates the status to “on\_the\_way,” and the system shares their live location. Once the vehicle is towed to the customer’s preferred garage, the mechanic marks the job as “completed.”

The customer initiates payment, and the backend verifies it through the payment gateway. If the customer later disputes the charge, the **admin panel** allows administrators to override the booking status or initiate a refund. The refund request flows to the payment gateway, which reverses the transaction. The backend updates the database, and the customer receives a notification confirming the refund.

### *Scenario 3: Admin Verification of Mechanics*

Before mechanics can appear in customer searches, they must be verified by administrators. A mechanic registers on the platform and uploads KYC documents. The admin panel displays the mechanic’s profile with a “verification pending” status. The administrator reviews the documents and toggles verification to “true.”

This update flows to the backend, which updates the `Mechanics` table. Verified mechanics are now visible to customers during booking searches. If a mechanic is later found to be fraudulent, the admin can revoke verification, hiding them from customer searches. This ensures trust and accountability in the system

## ER Diagram (Entity–Relationship Model Explanation)

The **Entity–Relationship (ER) Diagram** for the Insta Mechanic project visually represents how different entities (tables) in the database are connected. Since I can't render an actual image here, I'll describe the diagram in detail as if you were looking at it, so you can later draw or model it in tools like Lucidchart, Draw.io, or even on paper.

### Core Entities

#### 1. Users

- Attributes: `user_id` (PK), `name`, `email`, `phone`, `password_hash`, `role` (customer, mechanic, admin).
- This is the parent entity for all system users.

#### 2. Mechanics

- Attributes: `mechanic_id` (PK), `user_id` (FK), `verification_status`, `availability`, `location_lat`, `location_long`.
- Relationship: One **User** can be a **Mechanic** (1:1).

#### 3. Services

- Attributes: `service_id` (PK), `name`, `description`, `base_fee`.
- Relationship: One **Service** can be linked to many **Bookings** (1:N).

#### 4. Bookings

- Attributes: `booking_id` (PK), `customer_id` (FK), `mechanic_id` (FK), `service_id` (FK), `status` (requested, accepted, in\_progress, completed), `location_lat`, `location_long`, `preferred_time`.
- Relationships:
  - One **Customer (User)** can create many **Bookings** (1:N).
  - One **Mechanic** can be assigned to many **Bookings** (1:N).
  - One **Service** can be part of many **Bookings** (1:N).

#### 5. Payments

- Attributes: `payment_id` (PK), `booking_id` (FK), `amount`, `transaction_id`, `status` (pending, paid, refunded), `timestamp`.
- Relationship: One **Booking** has one **Payment** (1:1).

#### 6. Reviews (Enhancement Phase)

- Attributes: `review_id` (PK), `booking_id` (FK), `customer_id` (FK), `mechanic_id` (FK), `rating`, `comment`, `timestamp`.
- Relationship: One **Booking** can generate one **Review** (1:1).

#### 7. Vehicles (Enhancement Phase)

- **Attributes:** `vehicle_id` (PK), `customer_id` (FK), `make`, `model`, `year`, `registration_number`.
- **Relationship:** One **Customer** can save multiple **Vehicles** (1:N).

## Use Case Diagram

The **Use Case Diagram** for the Insta Mechanic project illustrates how different actors (customers, mechanics, and administrators) interact with the system and what functionalities they can access. It is a high-level representation of the system's behavior, showing the relationships between users and the services provided by the application.

### *Actors*

1. **Customer** – The end-user who books services, makes payments, and provides feedback.
2. **Mechanic** – The service provider who accepts jobs, updates status, and completes bookings.
3. **Administrator (Admin)** – The overseer who verifies mechanics, manages bookings, and handles disputes/refunds.

### *Customer Use Cases*

- **Register/Login:** Customers create accounts or log in securely.
- **Book Service:** Submit booking requests with details of vehicle and issue.
- **View Booking Status:** Track mechanic acceptance and job progress in real time.
- **Make Payment:** Complete transactions via Razorpay/Stripe.
- **Provide Review:** Rate and review mechanics after job completion.
- **Save Vehicle Details** (enhancement): Store vehicle information for faster future bookings.

### *Mechanic Use Cases*

- **Register/Login:** Mechanics create accounts and log in securely.
- **Update Availability:** Toggle availability to receive jobs.
- **Accept/Decline Booking:** Respond to customer requests.
- **Update Job Status:** Change booking status (`on_the_way`, `in_progress`, `completed`).
- **View Earnings Dashboard:** Track income and request payouts.

## *Admin Use Cases*

- **Verify Mechanics:** Approve or reject mechanic registrations based on KYC.
- **Monitor Bookings:** View all bookings and statuses.
- **Handle Disputes:** Override booking statuses or initiate refunds.
- **Manage Service Areas:** Configure pincodes and surge pricing for city expansion.

## Text-Based Use Case Diagram Representation

### Code

Actors:

Customer  
Mechanic  
Admin

Customer → (Register/Login)  
Customer → (Book Service)  
Customer → (View Booking Status)  
Customer → (Make Payment)  
Customer → (Provide Review)  
Customer → (Save Vehicle Details)

Mechanic → (Register/Login)  
Mechanic → (Update Availability)  
Mechanic → (Accept/Decline Booking)  
Mechanic → (Update Job Status)  
Mechanic → (View Earnings Dashboard)

Admin → (Verify Mechanics)  
Admin → (Monitor Bookings)  
Admin → (Handle Disputes)  
Admin → (Manage Service Areas)

## Narrative Flow

When a **customer** interacts with the system, they begin by registering or logging in. They then book a service, which creates a booking record in the backend. The booking status is updated in real time as mechanics accept and progress through the job. Once completed, the customer makes payment and optionally provides a review.

The **mechanic** interacts by logging in, updating availability, and accepting or declining bookings. They update job statuses as they progress, and their earnings dashboard reflects completed jobs and payouts.

# Sequence Flow for Booking System

The **sequence flow** of the Insta Mechanic booking system describes the step-by-step interactions between the customer, the frontend client, the backend server, the database, the mechanic, and external services such as payment gateways. This flow ensures that every booking request is processed consistently, securely, and transparently.

## *Step 1: Customer Initiates Booking*

The process begins when a **customer** opens the booking page and fills out the form with details such as vehicle type, issue description, location, and preferred time. Once the form is validated on the frontend, the data is packaged into a JSON request and sent to the backend API.

## *Step 2: Backend Receives Request*

The **backend server** receives the booking request through the `BookingController`. Authentication middleware checks the customer's JWT token to ensure they are logged in. If valid, the backend generates a unique booking ID, calculates the estimated cost, and stores the booking in the **Bookings table** of the PostgreSQL database with status set to "requested."

## *Step 3: Notification to Mechanics*

Once the booking is stored, the backend uses **Socket.io** to broadcast the new job request to all available mechanics within the service radius. The **mechanic dashboard** displays the booking details, including customer location and service type.

## *Step 4: Mechanic Accepts Job*

A mechanic reviews the request and clicks "Accept." The backend updates the booking status from "requested" to "accepted" and locks the record to prevent multiple mechanics from claiming the same job. This update is pushed back to the customer's frontend, where they see confirmation that a mechanic is on the way.

## *Step 5: Job Progress Updates*

As the mechanic proceeds, they update the booking status to "on\_the\_way," "in\_progress," and finally "completed." Each status change flows from the mechanic's dashboard to the backend, which validates the transition and updates the database. Socket.io pushes these updates to the customer's frontend in real time, ensuring transparency.

### *Step 6: Payment Processing*

Once the job is marked “completed,” the backend triggers the payment flow. The customer initiates payment through Razorpay/Stripe. The backend creates an order ID and sends it to the payment gateway. The gateway processes the transaction and sends a **webhook** back to the backend, confirming success or failure. The backend updates the **Payments table** accordingly and generates a receipt.

### *Step 7: Notifications & Records*

The backend sends a confirmation email to the customer with booking details and receipt using Nodemailer. The mechanic’s earnings dashboard is updated to reflect the new income. The admin panel also records the transaction, allowing administrators to monitor activity.

## **Sequence Flow for Payment System**

The **payment system sequence flow** in Insta Mechanic ensures that financial transactions are handled securely, transparently, and reliably. It involves interactions between the customer, frontend client, backend server, payment gateway (Razorpay/Stripe), database, and notification services.

### *Step 1: Payment Initiation*

Once a mechanic marks a booking as “completed,” the backend triggers the payment process. The **customer** sees a “Pay Now” button on their frontend interface. When clicked, the frontend sends a request to the backend to initiate payment.

### *Step 2: Backend Creates Order*

The **backend server** receives the request and generates a payment order using the Razorpay/Stripe API. This order includes details such as booking ID, customer ID, amount, and currency. The backend stores this order in the **Payments table** with status set to “pending.”

### *Step 3: Customer Completes Payment*

The backend returns the order ID to the frontend, which opens the payment gateway’s secure checkout modal. The **customer** enters card details, UPI, or wallet information. The payment gateway processes the transaction and provides immediate feedback to the customer.

#### *Step 4: Gateway Sends Webhook*

Once the transaction is processed, the **payment gateway** sends a **webhook** to the backend, confirming whether the payment was successful or failed. The webhook includes transaction ID, booking ID, and payment status.

#### *Step 5: Backend Updates Database*

The backend verifies the webhook signature to ensure authenticity. If valid, it updates the **Payments table**:

- Status changes from “pending” to “paid” if successful.
- Status changes to “failed” if unsuccessful. The backend also updates the booking record to reflect payment completion.

#### *Step 6: Notifications*

After updating the database, the backend sends a confirmation email to the **customer** with the receipt using Nodemailer. The mechanic’s earnings dashboard is updated to reflect the new income entry. The **admin panel** also records the transaction for monitoring and compliance.

#### **Text-Based Sequence Diagram Representation**

```
Customer → Frontend (Click Pay Now)
Frontend → Backend (API Request: Initiate Payment)
Backend → Payment Gateway (Create Order)
Backend → Database (Insert Payment Record: Pending)
Frontend → Payment Gateway (Customer Completes Payment)
Payment Gateway → Backend (Webhook Confirmation)
Backend → Database (Update Payment Record: Paid/Failed)
Backend → Customer (Email Receipt)
Backend → Mechanic (Update Earnings Dashboard)
Admin → Backend (Monitor Transactions)
```

## **Security Flow for Payment System**

The **security flow** of the Insta Mechanic payment system is designed to protect customer data, prevent fraud, and ensure that every transaction is verified before being recorded. This flow integrates multiple layers of security across the client, server, database, and payment gateway.

#### *Step 1: Secure Payment Initiation*

When a customer clicks “Pay Now,” the **frontend** enforces input validation and ensures that sensitive payment details are never stored locally. Instead, the frontend communicates with the backend using **HTTPS (SSL/TLS encryption)**, guaranteeing that all data in transit is protected against interception.



### *Step 2: Backend Order Creation*

The **backend server** generates a payment order using Razorpay/Stripe APIs. This order includes booking ID, customer ID, and amount. Before sending the order to the gateway, the backend attaches a **unique signature** and stores the order in the **Payments table** with status “pending.” This ensures that every transaction is traceable and linked to a booking.

### *Step 3: Customer Payment Entry*

The **customer** enters card details, UPI, or wallet information directly into the payment gateway’s secure checkout modal. Importantly, the backend never sees raw card data — this is handled entirely by the gateway, reducing PCI compliance risks.

### *Step 4: Gateway Verification*

The **payment gateway** processes the transaction and generates a **transaction ID**. It then sends a **webhook** to the backend, containing transaction details and a digital signature. The backend verifies this signature using the gateway’s public key, ensuring that the webhook is authentic and not tampered with.

### *Step 5: Database Update*

Once verified, the backend updates the **Payments table**:

- Status changes from “pending” to “paid” if successful.
- Status changes to “failed” if declined. The backend also updates the booking record to reflect payment completion. This ensures that no booking can be marked “completed” without a verified payment.

### *Step 6: Fraud Prevention*

Several fraud-prevention mechanisms are embedded:

- **Rate Limiting:** Prevents brute-force attempts on payment initiation.
- **JWT Authentication:** Ensures only logged-in customers can initiate payments.
- **Webhook Signature Verification:** Confirms authenticity of gateway responses.
- **Audit Logs:** Track every payment attempt, including failures, for compliance.
- **Short-Lived Tokens:** Reduce risk of replay attacks.

### *Step 7: Notifications & Transparency*

After successful payment, the backend generates a receipt and emails it to the customer. The mechanic’s earnings dashboard is updated, and the admin panel records the transaction. If fraud or disputes occur, admins can initiate refunds, which flow back through the gateway with full verification.

# Activity Flow of Service Lifecycle

The **service lifecycle in Insta Mechanic** follows a structured activity flow that ensures every booking moves smoothly from initiation to completion. This lifecycle captures the journey of a service request, detailing how customers, mechanics, administrators, and the system interact at each stage.

## *1. Service Request Initiation*

The lifecycle begins when a **customer** identifies a problem with their vehicle and submits a booking request through the frontend interface. The booking form captures essential details such as customer information, vehicle type, issue description, location, and preferred time. The frontend validates the inputs before sending them to the backend.

## *2. Booking Creation & Validation*

The **backend server** receives the request, authenticates the customer using JWT tokens, and generates a unique booking ID. The booking is stored in the **Bookings table** with status set to “requested.” At this stage, the system also calculates an estimated cost based on service type and distance.

## *3. Mechanic Notification & Acceptance*

Once the booking is created, the backend broadcasts the request to available mechanics using **Socket.io**. Mechanics receive notifications on their dashboard and can choose to accept or decline the job. When a mechanic accepts, the backend updates the booking status to “accepted” and locks the record to prevent duplicate claims. The customer is notified in real time that a mechanic is on the way.

## *4. Service Execution*

The mechanic begins the job and updates the booking status as they progress:

- **On\_the\_way**: Mechanic is traveling to the customer’s location.
- **In\_progress**: Mechanic is actively working on the vehicle.
- **Completed**: Service has been successfully delivered.

Each status update flows from the mechanic’s dashboard to the backend, which validates the transition and updates the database. Customers receive real-time updates through the frontend interface.

## *5. Payment Processing*

After completion, the system triggers the **payment lifecycle**. The customer initiates payment via Razorpay/Stripe. The backend creates an order, stores it in the **Payments**

**table** as “pending,” and sends the order ID to the frontend. The customer completes payment in the gateway’s secure checkout modal. The gateway sends a webhook to the backend, which verifies authenticity and updates the payment record to “paid.” A receipt is generated and emailed to the customer.

## 6. Review & Feedback

Once payment is confirmed, the customer is prompted to leave a **review**. They can rate the mechanic and provide comments. This review is stored in the **Reviews table**, linked to the booking and mechanic. Reviews contribute to the mechanic’s reputation system, improving transparency and trust.

## 7. Admin Oversight

Throughout the lifecycle, **administrators** oversee activities via the admin panel. They verify mechanics during registration, monitor bookings, and intervene in disputes. If a customer raises a complaint, admins can override booking statuses or initiate refunds. Refunds flow back through the payment gateway, with updates recorded in the database.

### Text-Based Activity Flow Diagram

```
Customer → Submit Booking Request
Frontend → Backend (Validate & Create Booking)
Backend → Database (Insert Booking: Requested)
Backend → Mechanics (Notify via Socket.io)
Mechanic → Backend (Accept Booking)
Backend → Database (Update Status: Accepted)
Backend → Customer (Realtime Confirmation)
Mechanic → Backend (Update Status: On_the_way → In_progress → Completed)
Backend → Database (Update Status)
Backend → Customer (Realtime Updates)
Customer → Payment Gateway (Initiate Payment)
Payment Gateway → Backend (Webhook Confirmation)
Backend → Database (Update Payment: Paid)
Backend → Customer (Email Receipt)
Customer → Backend (Submit Review)
Backend → Database (Insert Review)
Admin → Backend (Monitor/Override/Refund if needed)
```

## State Transition Diagram for Booking Lifecycle

The **state transition diagram** for the Insta Mechanic booking system illustrates how a booking moves through different states from initiation to completion, including exceptional cases like cancellation or refund. Each state represents a snapshot of the booking’s status, and transitions show the events that trigger movement between states.

### States in the Booking Lifecycle

#### 1. Requested

- Initial state when a customer submits a booking.
- Trigger: Customer fills booking form and submits.
- 2. **Accepted**
  - State when a mechanic accepts the booking.
  - Trigger: Mechanic clicks “Accept” on dashboard.
  - Constraint: Only one mechanic can accept; booking is locked.
- 3. **On\_the\_way**
  - Mechanic is traveling to the customer’s location.
  - Trigger: Mechanic updates status to “On\_the\_way.”
- 4. **In\_progress**
  - Mechanic is actively working on the vehicle.
  - Trigger: Mechanic updates status to “In\_progress.”
- 5. **Completed**
  - Service has been successfully delivered.
  - Trigger: Mechanic updates status to “Completed.”
- 6. **Paid**
  - Customer completes payment via Razorpay/Stripe.
  - Trigger: Payment gateway webhook confirms success.
- 7. **Refunded** (Exceptional State)
  - Admin initiates refund due to dispute or failure.
  - Trigger: Admin overrides payment status and processes refund.
- 8. **Cancelled** (Exceptional State)
  - Booking cancelled by customer or admin before acceptance.
  - Trigger: Customer cancels request or admin intervenes.

#### ***Text-Based State Transition Diagram***

Requested → Accepted → On\_the\_way → In\_progress → Completed → Paid  
Requested → Cancelled  
Completed → Paid → Refunded

# Activity Diagram for Payment Lifecycle

The **payment lifecycle activity diagram** in Insta Mechanic shows how a payment request moves through different stages, from initiation to completion, including exceptional cases like failure or refund. This diagram captures the activities performed by the customer, backend, payment gateway, and admin, ensuring that every transaction is secure and traceable.

## *Key Activities*

1. **Payment Initiation**
  - Triggered when a booking is marked “completed.”
  - Customer clicks **Pay Now** on the frontend.
  - Activity: Frontend sends request to backend to create payment order.
2. **Order Creation**
  - Backend generates a payment order with booking ID, customer ID, and amount.
  - Activity: Insert record into **Payments table** with status “pending.”
  - Backend sends order ID to frontend.
3. **Customer Payment Entry**
  - Customer enters card/UPI/wallet details in the gateway’s secure checkout modal.
  - Activity: Payment gateway processes transaction.
4. **Gateway Response**
  - Gateway sends **webhook** to backend with transaction details and signature.
  - Activity: Backend verifies signature for authenticity.
5. **Database Update**
  - If successful → Payment status updated to “paid.”
  - If failed → Payment status updated to “failed.”
  - Activity: Booking record updated accordingly.
6. **Notifications**
  - Backend sends receipt to customer via email.
  - Mechanic’s earnings dashboard updated.
  - Admin panel records transaction for monitoring.
7. **Refund (Exceptional Case)**
  - If dispute arises, admin initiates refund.
  - Activity: Backend communicates with gateway to reverse transaction.
  - Database updates payment status to “refunded.”
  - Customer notified via email.

## Use Case Scenarios

To make the **system design and data flow** more tangible, let's walk through real-world use case scenarios that demonstrate how the Insta Mechanic platform operates end-to-end. These scenarios highlight the interactions between customers, mechanics, administrators, and the backend system, showing how data moves through the architecture in practical situations.

### *Scenario 1: Battery Breakdown on the Road*

Imagine a customer driving late at night when their car battery suddenly fails. The customer opens the **Insta Mechanic website** on their mobile phone and navigates to the booking form. They enter their details — name, phone number, vehicle type, and select “Battery Replacement” as the issue. The system captures their location using the HTML5 Geolocation API and calculates an estimated cost based on the base fee plus distance.

This booking request flows to the backend, where the **BookingController** creates a new record in the `Bookings` table. A unique booking ID is generated, and the status is set to “requested.” Socket.io broadcasts this request to all available mechanics within the service radius. On the mechanic's dashboard, the job appears with details of the customer's location and issue.

One mechanic accepts the job, and the backend updates the booking status to “accepted.” This change is pushed back to the customer's frontend, where they see confirmation that a mechanic is on the way. The mechanic's location is periodically updated and shared via a Google Maps link, allowing the customer to track ETA. Once the mechanic arrives and replaces the battery, they update the status to “completed.” The backend verifies the transition, updates the database, and triggers a payment request.

The customer pays securely through Razorpay. The payment gateway processes the transaction and sends a webhook to the backend, which verifies authenticity and updates the `Payments` table. A receipt is generated, emailed to the customer, and stored in the database. The mechanic's earnings dashboard reflects the new income entry, closing the loop of this scenario.

### *Scenario 2: Towing Request After Accident*

A customer experiences a minor accident and needs towing services. They use the booking form to request “Towing Assistance.” The backend processes the request, stores it in the database, and broadcasts it to nearby mechanics with towing capability.

A mechanic with a tow truck accepts the job. The booking status changes to “accepted,” and the customer receives real-time updates. The mechanic updates the status to

“on\_the\_way,” and the system shares their live location. Once the vehicle is towed to the customer’s preferred garage, the mechanic marks the job as “completed.”

The customer initiates payment, and the backend verifies it through the payment gateway. If the customer later disputes the charge, the **admin panel** allows administrators to override the booking status or initiate a refund. The refund request flows to the payment gateway, which reverses the transaction. The backend updates the database, and the customer receives a notification confirming the refund.

### *Scenario 3: Admin Verification of Mechanics*

Before mechanics can appear in customer searches, they must be verified by administrators. A mechanic registers on the platform and uploads KYC documents. The admin panel displays the mechanic’s profile with a “verification pending” status. The administrator reviews the documents and toggles verification to “true.”

This update flows to the backend, which updates the `Mechanics` table. Verified mechanics are now visible to customers during booking searches. If a mechanic is later found to be fraudulent, the admin can revoke verification, hiding them from customer searches. This ensures trust and accountability in the system. Phase 0 — Planning & Design

#### **A. Objectives**

The planning and design phase is critical because it sets the foundation for the entire project. The objectives include:

- Defining the **problem statement**: Customers struggle to find verified mechanics quickly.
- Establishing the **solution**: A responsive web platform that connects customers, mechanics, and admins.
- Outlining the **scope**: MVP limited to one city, with core features of booking, mechanic acceptance, and admin verification.
- Identifying **out-of-scope items**: Advanced features like subscriptions, coupons, and multi-city expansion are deferred to later phases.

#### **B. Deliverables**

- **PRD (Product Requirements Document)**: A one-page document summarizing the problem, solution, and MVP scope.
- **User Stories**: Written from the perspective of each persona. For example:
  - *As a customer, I can request a mechanic by selecting a service and entering my location.*
  - *As a mechanic, I can go online/offline, accept a job, and update status.*
  - *As an admin, I can verify mechanics and view all bookings.*

- **Wireframes:** Low-fidelity sketches of the Home, Services, Booking, Mechanic Dashboard, and Admin Panel.
- **Repository Setup:** Decide between a mono-repo (frontend + backend together) or separate repos.

### C. Analysis

Planning ensures that the project avoids **scope creep** and maintains focus on MVP delivery. Wireframes help visualize the user journey, while user stories clarify functional requirements. The PRD acts as a contract between stakeholders, ensuring alignment.

### D. Challenges

- Balancing MVP scope with long-term vision.
- Ensuring wireframes are simple yet scalable.
- Avoiding over-engineering in the initial design.

### E. Acceptance Criteria

- PRD and user stories documented.
- Wireframes approved by stakeholders.
- Repository initialized with basic structure.

## V. Phase 1 — Frontend Bootstrap Prototype

### A. Objectives

To build a static, responsive prototype using **Bootstrap 5**, ensuring mobile-first design and accessibility.

### B. Deliverables

- **Pages:**
  - `index.html` (Home)
  - `services.html`
  - `book.html` (Booking form placeholder)
  - `about.html`, `contact.html`
- **UI Components:**
  - Navbar with logo and links.
  - Hero section with tagline and CTA button.
  - Services grid with cards (Battery, Tyre, Towing, General Service).
  - Footer with contact info and social links.



### C. Analysis

Bootstrap's grid system simplifies responsive design. Accessibility features like ARIA labels and keyboard navigation ensure inclusivity.

### D. Challenges

- Maintaining consistent design across devices.
- Ensuring fast load times with optimized assets.

### E. Acceptance Criteria

- Mobile-friendly static site.
- Lighthouse performance score > 80.
- No broken links.

## VI. Phase 2 — Booking Form & Validation

### A. Objectives

To create a functional booking form with client-side validation and a mock confirmation system.

### B. Deliverables

- **Form Fields:** Name, Phone, Email, Vehicle type, Issue type, Description, Address, Location (hidden lat/lng), Preferred time.
- **Validation:** Bootstrap validation states, regex for phone/email.
- **Cost Estimate Widget:** Static calculation per service.
- **Confirmation Screen:** Displays booking ID (mock, stored in LocalStorage).

### C. Analysis

Client-side validation prevents incomplete submissions. The cost estimate widget provides transparency, improving customer trust.

### D. Challenges

- Designing intuitive UX for form submission.
- Handling invalid inputs gracefully.

### E. Acceptance Criteria

- Valid form cannot submit empty.
- Estimate displayed correctly.

- Confirmation screen shows booking ID.

## Phase 3 — Backend API & Database

### A. Objectives

The backend phase establishes the **core server logic, authentication, and database persistence**. The goal is to move beyond static frontend prototypes and enable real data storage, retrieval, and secure user management.

Key objectives:

- Set up an **Express.js application** with RESTful APIs.
- Implement **JWT-based authentication** for customers, mechanics, and admins.
- Design and migrate a **PostgreSQL database schema**.
- Provide CRUD operations for services, bookings, and mechanic availability.
- Ensure secure password handling and role-based access control.

### B. Deliverables

- **Express App:** `/backend/src/index.js` with health route (`GET /api/health`).
- **Database Connection:** PostgreSQL via Prisma ORM or Sequelize.
- **Authentication:**
  - `POST /api/auth/register` (role: customer | mechanic | admin).
  - `POST /api/auth/login` (JWT issued).
- **Core Endpoints:**
  - `GET /api/services` — fetch available services.
  - `GET /api/mechanics/nearby?lat&lng&serviceId` — query mechanics using Haversine formula.
  - `POST /api/bookings` — create booking request.
  - `GET /api/bookings/:id` — retrieve booking details.
  - `PATCH /api/bookings/:id/status` — update booking status (requested → accepted → on\_the\_way → in\_progress → completed → cancelled).
  - `PATCH /api/me/profile` — update mechanic/customer profile.
  - `PATCH /api/mechanics/:id/availability` — toggle online/offline.
- **Testing Tools:** Postman/Insomnia collections for API testing.

### C. Database Schema Design

#### Tables

1. **Users**
  - `id` (PK)
  - `name`
  - `email`
  - `phone`

- password\_hash
- role (ENUM: customer, mechanic, admin)
- created\_at, updated\_at
- 2. **Mechanics**
  - id (PK)
  - user\_id (FK → Users.id)
  - verified (BOOLEAN)
  - availability (BOOLEAN)
  - location\_lat, location\_lng
  - services\_offered (ARRAY)
- 3. **Services**
  - id (PK)
  - name (Battery, Tyre, Towing, etc.)
  - base\_price
- 4. **Bookings**
  - id (PK)
  - customer\_id (FK → Users.id)
  - mechanic\_id (FK → Mechanics.id)
  - service\_id (FK → Services.id)
  - status (ENUM)
  - description
  - address
  - location\_lat, location\_lng
  - preferred\_time
  - paid (BOOLEAN)
  - created\_at, updated\_at
- 5. **Payments**
  - id (PK)
  - booking\_id (FK → Bookings.id)
  - amount
  - transaction\_id
  - status (ENUM: pending, success, failed)
  - created\_at

## D. Technical Analysis

- **Authentication:** JWT tokens ensure stateless sessions. Passwords are hashed using **bcrypt**. Tokens have short expiry to reduce risk.
- **Nearby Mechanics Query:** Uses **Haversine formula** to calculate distance between customer and mechanic coordinates.
- **Role-Based Access Control:** Middleware checks JWT and role before allowing access to endpoints.
- **Error Handling:** Standardized JSON error responses with status codes (400, 401, 404, 500).
- **Scalability:** PostgreSQL chosen for relational consistency; Prisma ORM simplifies migrations and queries.

## E. Challenges

- **Security Risks:** JWT token theft, SQL injection, weak password policies.

- **Concurrency:** Multiple mechanics accepting the same job simultaneously.
- **Data Integrity:** Ensuring bookings cannot be updated after completion.
- **Performance:** Optimizing geolocation queries for large datasets.

## F. Acceptance Criteria

- API returns JSON responses consistently.
- JWT authentication works across roles.
- Database persists users, bookings, and payments.
- Postman tests pass for all endpoints.

## Example Endpoint Flow

### Booking Creation (POST /api/bookings)

1. Customer submits booking form.
2. Backend validates JWT and input.
3. Booking record created in PostgreSQL.
4. Response: { bookingId: 123, status: "requested" }.

### Mechanic Acceptance (PATCH /api/bookings/:id/status)

1. Mechanic toggles availability.
2. Mechanic accepts booking → status updated to `accepted`.
3. Customer notified via frontend.

## Phase 4 — Mechanic Dashboard

### A. Objectives

The mechanic dashboard is designed to provide mechanics with a **centralized interface** to manage their work. Its objectives are:

- Allow mechanics to **toggle availability** (online/offline).
- Display **assigned jobs** with details such as customer info, service type, and location.
- Enable mechanics to **accept or decline jobs**.
- Provide functionality to **update job status** (accepted → on\_the\_way → in\_progress → completed).
- Show **earnings summary** for completed jobs.

## B. Deliverables

- **Frontend Page:** `mechanic-dashboard.html` or integrated SPA route `/dashboard`.
- **UI Components:**
  - Availability toggle (switch button).
  - Job list (Bootstrap cards or tables).
  - Status update buttons (Accept, Decline, Progress).
  - Earnings section (weekly/monthly totals).
- **Backend Integration:**
  - `PATCH /api/mechanics/:id/availability` — toggle online/offline.
  - `GET /api/bookings?mechanicId=123` — fetch assigned jobs.
  - `PATCH /api/bookings/:id/status` — update job status.
- **Role-Based Rendering:** Dashboard only visible if `role = mechanic`.

## C. Technical Analysis

1. **Availability Toggle**
  - Implemented as a switch button in the UI.
  - Backend updates mechanic's `availability` field in PostgreSQL.
  - Customers only see mechanics marked as `availability = true`.
2. **Job List**
  - Fetched via API (`GET /api/bookings`).
  - Displayed in a responsive Bootstrap table with filters (pending, in-progress, completed).
  - Each job card includes customer name, service type, address, and preferred time.
3. **Status Updates**
  - Mechanics progress jobs through predefined states.
  - Backend ensures valid transitions (cannot skip from requested → completed directly).
  - Socket.io broadcasts status updates to customers in real time.
4. **Earnings Summary**
  - Calculated from completed bookings.
  - Backend aggregates payments linked to mechanic's ID.
  - Displayed as charts (weekly/monthly totals).

## D. Challenges

- **Concurrency:** Multiple mechanics may attempt to accept the same job. Solution: backend locks booking once accepted.
- **Data Consistency:** Preventing mechanics from updating jobs they are not assigned to. Solution: role-based JWT checks.
- **UI Complexity:** Ensuring dashboard remains simple while showing detailed job info.

- **Realtime Updates:** Handling Socket.io events efficiently without overwhelming the server.

## E. Acceptance Criteria

- Mechanic can toggle availability successfully.
- Assigned jobs appear correctly in dashboard.
- Status updates reflect in both mechanic and customer views.
- Earnings summary displays accurate totals.

## F. Example Workflow

1. Mechanic logs in → JWT verifies role = mechanic.
2. Mechanic toggles availability → backend updates DB.
3. Customer books service → backend assigns mechanic.
4. Mechanic sees job in dashboard → clicks “Accept.”
5. Status updates to `accepted` → customer notified in real time.
6. Mechanic progresses job → `on_the_way` → `in_progress` → `completed`.
7. Earnings updated automatically after payment confirmation.

# Phase 5 — Admin Panel

## A. Objectives

The Admin Panel is designed to provide administrators with **oversight and governance** over the platform. Its objectives are:

- Allow admins to **verify mechanics** through KYC (Know Your Customer) checks.
- Provide visibility into **all bookings** across the system.
- Enable **manual status overrides** in case of disputes or errors.
- Facilitate **refunds and adjustments** for failed or cancelled bookings.
- Offer **basic filters and search functionality** to manage large datasets.

## B. Deliverables

- **Frontend Page:** `admin/index.html` built with Bootstrap tables and badges.
- **UI Components:**
  - Mechanics table with verification toggle.
  - Bookings table with filters (status, date, customer, mechanic).
  - Badges for booking status (requested, accepted, completed, cancelled).
  - Search bar and pagination controls.
- **Backend Integration:**
  - `GET /api/admin/mechanics` — list mechanics with verification status.
  - `PATCH /api/admin/mechanics/:id/verify` — toggle verification.
  - `GET /api/admin/bookings` — list all bookings.

- PATCH /api/admin/bookings/:id/status — override booking status.
  - POST /api/admin/refunds — initiate refund process.
- **Role-Based Access Control:** Only users with `role = admin` can access these endpoints.

## C. Technical Analysis

1. **Mechanic Verification**
  - Admins review mechanic profiles and documents.
  - Verification status stored in `mechanics.verified = true/false`.
  - Verified mechanics appear in customer searches; unverified mechanics are hidden.
2. **Bookings Oversight**
  - Admins can view all bookings in a tabular format.
  - Filters allow sorting by date, status, or service type.
  - Pagination ensures scalability when handling thousands of records.
3. **Manual Status Override**
  - Admins can update booking status in case of disputes.
  - Example: If a mechanic fails to mark a job as completed, admin can override.
  - Overrides logged for audit purposes.
4. **Refunds Management**
  - Admins initiate refunds via payment gateway API.
  - Refund records linked to booking ID for traceability.
  - Customers notified via email upon refund initiation.

## D. Challenges

- **Data Integrity:** Preventing admins from accidentally overriding valid statuses.
- **Security Risks:** Ensuring only authorized admins can access sensitive data.
- **Scalability:** Handling large datasets efficiently with server-side pagination.
- **Auditability:** Maintaining logs of all admin actions for compliance.

## E. Acceptance Criteria

- Admin can verify mechanics successfully.
- Admin can view and filter bookings list.
- Admin can override booking status when necessary.
- Refunds processed correctly and linked to bookings.

## F. Example Workflow

1. Admin logs in → JWT verifies `role = admin`.
2. Admin navigates to Mechanics table → reviews mechanic profile.
3. Admin toggles verification → mechanic marked as verified.
4. Admin views Bookings table → filters by “pending” status.

5. Admin overrides booking status to “completed” after dispute resolution.
6. Admin initiates refund → payment gateway processes refund → customer notified.

## Phase 6 — Realtime, Location & Notifications

### A. Objectives

The goal of this phase is to integrate **real-time communication, geolocation tracking, and notifications** into the system. This ensures that customers and mechanics can see live updates of booking status, track location, and receive timely alerts.

Key objectives:

- Capture **customer and mechanic location** using HTML5 Geolocation API.
- Implement **Socket.io** for real-time booking status updates.
- Embed maps for **mechanic ETA visualization**.
- Send **notifications** via email and SMS when booking status changes.

### B. Deliverables

- **Frontend Enhancements:**
  - Geolocation capture on booking form.
  - Mechanic dashboard with periodic location updates.
  - Progress bar showing booking status (Requested → Accepted → On the way → In progress → Completed).
- **Backend Enhancements:**
  - Socket.io server integrated with Express.
  - WebSocket events: `booking:new`, `booking:update`, `mechanic:location`, `customer:location`.
  - Map embed using Google Maps link (initial MVP).
- **Notifications:**
  - Email alerts via Nodemailer.
  - Optional SMS alerts using Twilio or similar service.

### C. Technical Analysis

1. **Location Capture**
  - Customers: Location captured during booking submission.
  - Mechanics: Location updated periodically when online.
  - Stored in PostgreSQL with latitude/longitude fields.
2. **Realtime Updates (Socket.io)**
  - Socket.io establishes persistent connections between client and server.



- Events broadcast booking status changes instantly.
- Customers see live updates without page refresh.
- 3. **Map Embedding**
  - Initial MVP uses Google Maps link with mechanic coordinates.
  - Future enhancement: Full map embed with live tracking.
- 4. **Notifications**
  - Email notifications sent when booking status changes.
  - Example: “Your mechanic is on the way.”
  - SMS optional for urgent alerts.

## D. Challenges

- **Accuracy of Geolocation:** HTML5 Geolocation depends on device GPS/Wi-Fi, which may be imprecise.
- **Scalability of Socket.io:** Handling thousands of concurrent connections requires load balancing.
- **Notification Reliability:** Ensuring emails/SMS are delivered promptly and not flagged as spam.
- **Privacy Concerns:** Storing location data securely and allowing users to delete it.

## E. Acceptance Criteria

- Customer location captured during booking.
- Mechanic location updated periodically when online.
- Booking status updates visible in real time.
- Notifications sent successfully on status changes.

## F. Example Workflow

1. Customer submits booking → location captured.
2. Mechanic goes online → dashboard starts sending location updates.
3. Admin assigns mechanic → Socket.io broadcasts `booking:new`.
4. Mechanic accepts job → status changes to `accepted`.
5. Customer receives email: “Mechanic has accepted your booking.”
6. Mechanic updates status to `on_the_way` → customer sees progress bar update.
7. Mechanic location shared via Google Maps link → customer tracks ETA.
8. Job completed → customer notified via email/SMS.

# Phase 7 — Pricing & Payments

## A. Objectives

The pricing and payments phase ensures that customers are charged fairly and mechanics are compensated reliably. Its objectives are:

- Define a **pricing model** that balances customer affordability and mechanic earnings.
- Integrate **payment gateways (Razorpay/Stripe)** for secure transactions.
- Implement **server-side order creation and webhook verification**.
- Update booking records with payment status (`paid = true`).
- Provide customers with **receipts and transaction history**.

## B. Deliverables

- **Pricing Model:**
  - Base fee per service (e.g., Battery replacement ₹500).
  - Additional charges based on distance/time (simple formula for MVP).
- **Payment Gateway Integration:**
  - `POST /api/payments/create-order` — creates checkout order.
  - `POST /api/webhooks/razorpay` — verifies payment capture server-to-server.
- **Frontend Enhancements:**
  - Payment button on booking confirmation screen.
  - Razorpay/Stripe checkout modal embedded.
- **Backend Enhancements:**
  - Payment record linked to booking ID.
  - Webhook updates booking status to `paid = true`.
- **Receipts:**
  - Stored in database.
  - Sent via email to customer.

## C. Technical Analysis

1. **Pricing Model**
  - Formula: `total = base_fee + (distance * rate_per_km) + (time * rate_per_hour)`.
  - Simple calculation for MVP; scalable to surge pricing later.
2. **Payment Gateway Integration**
  - Razorpay/Stripe SDK integrated into backend.
  - Server creates order with amount and booking ID.
  - Customer completes payment via secure checkout modal.
3. **Webhook Verification**
  - Payment gateway sends webhook to backend after transaction.
  - Backend verifies signature to prevent fraud.
  - Booking record updated with `paid = true`.
4. **Receipts & History**
  - Receipt generated with booking ID, service, amount, and transaction ID.
  - Stored in PostgreSQL for audit.
  - Sent to customer via email.

## D. Challenges

- **Failed Transactions:** Handling retries and customer communication.
- **Fraud Prevention:** Ensuring webhook signatures are verified.
- **Currency Support:** Razorpay for INR, Stripe for global currencies.
- **Scalability:** Managing high transaction volumes during peak demand.

## E. Acceptance Criteria

- Successful payment updates booking record (`paid = true`).
- Receipt stored in database and emailed to customer.
- Webhook verification prevents fraudulent updates.
- Pricing model applied consistently across services.

## F. Example Workflow

1. Customer submits booking → cost estimate displayed.
2. Customer clicks “Pay Now” → Razorpay checkout modal opens.
3. Customer completes payment → Razorpay sends webhook to backend.
4. Backend verifies transaction → updates booking record (`paid = true`).
5. Receipt generated → stored in DB → emailed to customer.
6. Mechanic dashboard shows job as “paid” → mechanic proceeds with service.

# Phase 8 — Quality, Security & Documentation

## A. Objectives

The purpose of this phase is to guarantee that the system meets **quality standards**, adheres to **security best practices**, and is supported by **comprehensive documentation**. The objectives include:

- Implementing **unit, integration, and end-to-end testing**.
- Enforcing **security measures** to protect user data and prevent attacks.
- Establishing **error handling and logging mechanisms**.
- Producing **developer documentation** (README, API docs, ERD diagrams).

## B. Deliverables

- **Testing Suite:**
  - Unit tests using **Jest** for controllers and services.
  - API tests using **Supertest** for endpoints.
  - End-to-end (E2E) tests using **Playwright** for booking flow.
- **Security Enhancements:**

- Input validation with **zod/joi**.
  - Rate limiting on authentication and booking endpoints.
  - Helmet middleware for HTTP headers.
  - CORS configuration for cross-origin requests.
- **Error Handling & Logging:**
  - Structured JSON error responses (4xx/5xx).
  - Logging with **Winston/Pino**.
  - Monitoring failed requests and server errors.
- **Documentation:**
  - README with setup instructions.
  - `.env.example` file for environment variables.
  - ERD diagram of database schema.
  - API documentation using **OpenAPI/Swagger**.

## C. Technical Analysis

1. **Testing Strategy**
  - **Unit Tests:** Validate individual functions (e.g., password hashing, JWT generation).
  - **API Tests:** Ensure endpoints return correct responses and enforce authentication.
  - **E2E Tests:** Simulate full booking flow (customer books → mechanic accepts → job completed).
  - **Load Testing:** Using **k6** to simulate 50+ requests per second on `/bookings` endpoint.
2. **Security Measures**
  - **Password Hashing:** Using **bcrypt** with salt rounds.
  - **JWT Expiry:** Short-lived tokens with optional refresh tokens.
  - **Rate Limiting:** Prevent brute-force login attempts.
  - **Helmet Middleware:** Adds secure HTTP headers.
  - **CORS:** Restricts API access to trusted domains.
3. **Error Handling & Logging**
  - Standardized error format:

```

json

{ "error": "Invalid input", "code": 400 }
```
  - Logging with **Winston/Pino** for structured logs.
  - Error tracking integrated with monitoring tools (e.g., **Sentry**).
4. **Documentation**
  - README includes installation, run scripts, and deployment steps.
  - `.env.example` lists required environment variables (`DB_URL`, `JWT_SECRET`, `PAYMENT_KEYS`).
  - ERD diagram illustrates relationships between Users, Mechanics, Services, Bookings, and Payments.

- OpenAPI spec provides machine-readable API documentation.

## D. Challenges

- **Test Coverage:** Ensuring all critical paths are tested without slowing development.
- **Security Trade-offs:** Balancing usability (e.g., session length) with strict security.
- **Error Transparency:** Providing clear error messages without exposing sensitive details.
- **Documentation Maintenance:** Keeping docs updated as features evolve.

## E. Acceptance Criteria

- All unit, API, and E2E tests pass in CI pipeline.
- Security measures implemented and verified.
- Error handling produces clear, consistent JSON responses.
- Documentation complete and accessible to developers.

## F. Example Workflow

1. Developer runs `npm test` → Jest executes unit tests.
2. CI pipeline runs API tests → verifies endpoints return correct responses.
3. Playwright simulates booking flow → ensures customer and mechanic workflows succeed.
4. Load test with k6 → system handles 50+ requests per second.
5. Documentation reviewed → new developer can set up project in <30 minutes.

# Phase 9 — Deploy & Release

## A. Objectives

The deployment and release phase ensures that the **Insta Mechanic MVP** is accessible to the public with proper configuration, monitoring, and analytics. Its objectives are:

- Deploy the **frontend** to a static hosting service (Netlify/Vercel).
- Deploy the **backend** to a cloud platform (Render/Railway/Heroku).
- Host the **PostgreSQL database** on Railway or Supabase.
- Configure **environment variables** for secure operations.
- Set up **domain + SSL certificates** for secure access.
- Implement **monitoring and analytics** to track uptime and user activity.

## B. Deliverables

- **Frontend Deployment:**

- Netlify/Vercel hosting for static HTML/CSS/JS files.
  - Continuous deployment linked to GitHub repository.
- **Backend Deployment:**
  - Render/Railway hosting for Node.js/Express server.
  - Auto-deploy on push to main branch.
- **Database Hosting:**
  - PostgreSQL instance on Railway/Supabase.
  - Migrations run and services seeded.
- **Environment Variables:**
  - DB\_URL, JWT\_SECRET, PAYMENT\_KEYS, SMTP\_CREDS.
- **Domain & SSL:**
  - Custom domain configured.
  - SSL certificate applied for HTTPS.
- **Monitoring & Analytics:**
  - UptimeRobot for server monitoring.
  - Google Analytics for frontend activity tracking.

## C. Technical Analysis

1. **Frontend Deployment**
  - Netlify/Vercel chosen for simplicity and free tier support.
  - CI/CD pipeline ensures automatic deployment on code changes.
  - Lighthouse audits run post-deployment to verify performance.
2. **Backend Deployment**
  - Render/Railway provides containerized hosting for Node.js apps.
  - Environment variables injected securely.
  - Logs accessible via dashboard for debugging.
3. **Database Hosting**
  - Railway/Supabase offers managed PostgreSQL with backups.
  - Migrations applied via Prisma/Sequelize.
  - Seed data includes default services (Battery, Tyre, Towing).
4. **Environment Variables**
  - Stored securely in hosting platform.
  - `.env.example` provided for developers.
  - Secrets never committed to GitHub.
5. **Monitoring & Analytics**
  - UptimeRobot pings backend every 5 minutes to ensure availability.
  - Google Analytics tracks page views, bounce rates, and user flow.
  - Error tracking integrated with Sentry for frontend/backend issues.

## D. Challenges

- **Configuration Errors:** Incorrect environment variables can break deployment.
- **Database Connectivity:** Ensuring backend connects securely to hosted DB.
- **SSL Setup:** Avoiding mixed content errors when serving HTTPS.
- **Monitoring Overhead:** Balancing detailed logs with performance.

## E. Acceptance Criteria

- Public URL accessible with custom domain.
- End-to-end booking flow works in production.
- SSL certificate active; site loads over HTTPS.
- Monitoring tools confirm uptime > 99%.

## F. Example Workflow

1. Developer pushes code to GitHub → CI/CD triggers deployment.
2. Netlify builds frontend → deploys to public domain.
3. Render builds backend → connects to Railway PostgreSQL.
4. Environment variables configured (DB\_URL, JWT\_SECRET).
5. UptimeRobot confirms backend uptime.
6. Customer visits public URL → books mechanic → completes payment.
7. Admin verifies mechanic → booking record visible in production DB.

# Phase 10 — Feedback & Iteration

## A. Objectives

The feedback and iteration phase ensures that the **Insta Mechanic MVP** improves continuously after its initial release. Its objectives are:

- Conduct a **soft launch** to a limited audience (classmates, friends, or pilot users).
- Collect **structured feedback** on usability, performance, and reliability.
- Identify and fix **bugs and UI/UX issues**.
- Refine **pricing models and service workflows** based on user behavior.
- Prepare for **scaling and enhancements** in later phases.

## B. Deliverables

- **Feedback Collection Tools:**
  - Google Forms for structured surveys.
  - Built-in feedback widget on the website.
  - Direct interviews or focus groups with pilot users.
- **Bug Tracking System:**
  - GitHub Issues or Jira board for logging bugs.
  - Categorization: Critical, Major, Minor.
- **Iteration Plan:**
  - Weekly sprints to address feedback.
  - Prioritization of fixes over new features.
- **Refined UI Copy:**
  - Adjust wording on buttons, alerts, and notifications for clarity.
- **Pricing Adjustments:**
  - Modify base fees or distance/time multipliers based on feedback.

## C. Technical Analysis

1. **Soft Launch Strategy**
  - Limited release to a small group of trusted users.
  - Monitor system performance under real-world conditions.
  - Collect qualitative and quantitative feedback.
2. **Feedback Collection**
  - Google Forms: Structured questions on usability, speed, and satisfaction.
  - Built-in widget: Quick star ratings and comments.
  - Admin dashboard: Logs feedback linked to user accounts.
3. **Bug Fixing & Iteration**
  - Critical bugs (e.g., payment failures) fixed immediately.
  - Minor UI issues scheduled for later sprints.
  - Continuous integration ensures fixes are deployed quickly.
4. **UI/UX Refinement**
  - Simplify booking form fields based on user confusion.
  - Improve dashboard layout for mechanics.
  - Add tooltips and help text where needed.
5. **Pricing Adjustments**
  - Analyze booking data to identify underpriced/overpriced services.
  - Adjust formulas for distance/time charges.
  - Consider promotional discounts for early adopters.

## D. Challenges

- **User Engagement:** Ensuring enough feedback is collected during soft launch.
- **Prioritization:** Balancing bug fixes with feature requests.
- **Bias in Feedback:** Early users may be friends/classmates, leading to skewed opinions.
- **Iteration Speed:** Implementing changes quickly without breaking existing features.

## E. Acceptance Criteria

- At least 20–30 pilot users provide feedback.
- Critical bugs identified and resolved.
- UI copy refined based on feedback.
- Pricing adjusted to reflect real-world usage.

## F. Example Workflow

1. Project deployed publicly → shared with classmates/friends.
2. Users complete bookings → provide feedback via Google Form.
3. Admin reviews feedback → logs issues in GitHub/Jira.
4. Development team prioritizes fixes → deploys updates weekly.



5. UI copy adjusted → “Book Mechanic” button changed to “Request Service” for clarity.
6. Pricing model refined → base fee increased slightly to cover mechanic costs.
7. Iterated version released → feedback cycle continues.

## Phase 11 — Enhancements

### A. Objectives

The enhancement phase focuses on **adding advanced features** that improve user experience, increase engagement, and prepare the platform for scaling. Its objectives are:

- Introduce **ratings and reviews** for mechanics.
- Implement **coupons, referrals, and loyalty programs**.
- Add **saved vehicles and service history** for customers.
- Provide mechanics with an **earnings dashboard** and payout reports.
- Expand to **multiple cities** with configurable service areas and surge pricing.

### B. Deliverables

- **Ratings & Reviews:**
  - Customers rate mechanics after job completion.
  - Reviews stored in database and displayed on mechanic profiles.
- **Coupons & Referrals:**
  - Coupon codes for discounts.
  - Referral system where customers earn credits for inviting friends.
- **Saved Vehicles & Service History:**
  - Customers save vehicle details for faster bookings.
  - Service history log for tracking past repairs.
- **Mechanic Earnings Dashboard:**
  - Weekly/monthly earnings reports.
  - Payout requests integrated with payment gateway.
- **City Expansion Config:**
  - Admin defines serviceable pincodes.
  - Surge pricing applied during peak demand.

### C. Technical Analysis

1. **Ratings & Reviews**
  - Schema: `reviews` table with `customer_id`, `mechanic_id`, `rating`, `comment`.
  - Average rating displayed on mechanic profile.
  - Moderation tools for admins to remove inappropriate reviews.
2. **Coupons & Referrals**
  - Coupons stored in `coupons` table with expiry date and discount percentage.

- Referral codes linked to customer accounts.
- Backend applies discounts during payment calculation.
- 3. **Saved Vehicles & Service History**
  - Vehicles stored in `vehicles` table linked to customer ID.
  - Service history generated automatically after each booking.
  - Customers view past repairs and invoices.
- 4. **Mechanic Earnings Dashboard**
  - Backend aggregates completed bookings and payments.
  - Charts display weekly/monthly earnings.
  - Payout requests trigger Razorpay/Stripe transfer API.
- 5. **City Expansion Config**
  - Admin defines serviceable pincodes in `service_areas` table.
  - Surge pricing formula:  $\text{total} = \text{base\_fee} + (\text{distance} * \text{rate}) + \text{surge\_multiplier}$ .
  - Customers outside service area see “Service not available.”

## D. Challenges

- **Review Authenticity:** Preventing fake reviews or biased ratings.
- **Coupon Abuse:** Ensuring coupons are not exploited repeatedly.
- **Data Management:** Handling large volumes of service history records.
- **Payment Compliance:** Ensuring payouts comply with financial regulations.
- **Scalability:** Expanding to multiple cities without performance degradation.

## E. Acceptance Criteria

- Customers can rate and review mechanics after job completion.
- Coupons and referrals applied correctly during payment.
- Customers can save vehicles and view service history.
- Mechanics can view accurate earnings reports.
- Admin can configure service areas and surge pricing.

## F. Example Workflow

1. Customer completes booking → prompted to rate mechanic.
2. Customer enters rating and comment → stored in DB → displayed on mechanic profile.
3. Customer applies coupon code during payment → discount applied.
4. Customer saves vehicle details → next booking auto-fills vehicle info.
5. Mechanic views dashboard → sees weekly earnings chart.
6. Admin expands service to new city → defines pincodes → surge pricing enabled.

# Tech Stack of Insta Mechanic

## 1. Frontend (Client Layer)

- **Languages & Frameworks:**
  - **HTML5** → Provides the structural foundation of the web application. It supports semantic elements (like `<header>`, `<section>`, `<footer>`) that improve accessibility and SEO.
  - **CSS3** → Used for styling, animations, and responsive layouts. Features like Flexbox and Grid ensure the UI adapts to different screen sizes (mobile, tablet, desktop).
  - **Bootstrap** → A CSS framework that accelerates development with prebuilt components (buttons, modals, forms). It ensures consistency across the interface.
  - **JavaScript (ES6)** → Adds interactivity (form validation, dynamic updates). ES6 features like arrow functions, promises, and `async/await` improve code readability and performance.
- **Hosting:**
  - **Netlify/Vercel** → These platforms provide global CDN distribution, meaning static assets are cached at edge servers worldwide. This reduces latency and ensures fast page loads. They also offer continuous deployment pipelines linked to GitHub/GitLab, so every code push automatically updates the live site.
- **Role in System:**
  - Customer booking interface.
  - Mechanic dashboard for job acceptance and progress updates.
  - Admin panel for monitoring and dispute resolution.

## 2. Backend (Application Layer)

- **Framework:**
  - **Node.js** → A runtime environment that executes JavaScript on the server side. It's event-driven and non-blocking, making it ideal for handling multiple concurrent requests.
  - **Express.js** → A lightweight web framework for Node.js. It simplifies routing, middleware management, and API creation.
- **Responsibilities:**
  - API endpoints for booking, payments, reviews, authentication.
  - Real-time communication using **Socket.io** (WebSocket-based library).
  - Business logic controllers (e.g., `AuthController`, `BookingController`, `PaymentController`).
- **Hosting:**
  - **Render/Railway/Heroku** → Cloud platforms that support containerized deployment. They provide auto-scaling, load balancing, and environment variable management for secure API keys.
- **Role in System:**

- Acts as the “brain” of Insta Mechanic, processing requests, applying business rules, and coordinating between frontend, database, and external services.

### 3. Database (Data Layer)

- **Database:**
  - **PostgreSQL** → A relational database known for reliability, ACID compliance, and advanced features like JSON support.
- **Hosting:**
  - **Supabase/Railway/AWS RDS** → Managed database services that provide automated backups, replication, and scaling.
- **Schema Design:**
  - **Users Table** → Stores customer and admin details.
  - **Mechanics Table** → Stores mechanic profiles, verification status, and ratings.
  - **Bookings Table** → Tracks service requests, statuses, timestamps.
  - **Payments Table** → Records transactions, statuses (pending, paid, refunded).
  - **Reviews Table** → Stores customer feedback linked to bookings.
  - **Vehicles Table** → Stores vehicle details for predictive maintenance.
- **Optimization Techniques:**
  - Indexing on frequently queried fields (e.g., booking status, mechanic location).
  - Partitioning large tables (e.g., bookings by month).
  - Normalization to reduce redundancy.

### 4. External Services Integration

- **Payment Gateways:** Razorpay/Stripe → Handle secure transactions, provide webhook notifications for payment status.
- **Email Notifications:** Nodemailer → Sends booking confirmations, receipts, and mechanic updates.
- **SMS Alerts:** Twilio → Provides urgent notifications for breakdowns or service updates.
- **Maps API:** Google Maps → Enables location visualization, distance calculation, and ETA tracking.
- **Role in System:** Extends functionality without reinventing core services, ensuring reliability and compliance.

### 5. Security Layer

- **Frontend Security:** HTTPS enforced, client-side input validation.
- **Backend Security:**
  - **JWT Authentication** → Ensures only authorized users access APIs.
  - **bcrypt** → Hashes passwords securely.

- **Helmet Middleware** → Adds secure HTTP headers to prevent attacks.
  - **CORS Policies** → Restrict API access to trusted domains.
- **Database Security:** SSL encryption, role-based access control.
- **Fraud Prevention:** Rate limiting, webhook signature verification, audit logs.

## Authentication (Auth)

Authentication (often shortened to **Auth**) is one of the most critical components of the Insta Mechanic system. It ensures that only legitimate users (customers, mechanics, and admins) can access the platform's services securely. Let's break down the **Auth architecture** in detail:

### 1. Purpose of Authentication

- **Identity Verification:** Confirms that the user is who they claim to be.
- **Access Control:** Grants different permissions to customers, mechanics, and admins.
- **Security:** Protects sensitive data (payments, bookings, reviews).
- **Trust:** Builds confidence among users that their accounts and transactions are safe.

### 2. Authentication Flow

1. **User Registration**
  - Customers and mechanics register with email, phone number, and password.
  - Mechanics undergo an additional **verification process** (admin approval, document upload).
  - Passwords are hashed using **bcrypt** before being stored in the database.
2. **Login Process**
  - User enters credentials (email/phone + password).
  - Backend verifies credentials against the database.
  - If valid, backend generates a **JWT (JSON Web Token)**.
3. **Token Issuance**
  - JWT contains user ID, role (customer/mechanic/admin), and expiry time.
  - Token is signed with a secret key to prevent tampering.
  - Token is sent to the frontend and stored securely (in HTTP-only cookies or local storage).
4. **Protected Routes**
  - When a user makes a request (e.g., booking a service), the frontend attaches the JWT in the request header.
  - Backend middleware verifies the token before granting access.
  - If token is invalid/expired, access is denied.

### 3. Role-Based Authentication

- **Customer Role:**
  - Can book services, make payments, leave reviews.
- **Mechanic Role:**
  - Can accept jobs, update progress, view earnings.
- **Admin Role:**
  - Can verify mechanics, monitor bookings, resolve disputes, initiate refunds.

Each role has **different permissions**, enforced by backend middleware.

### 4. Security Features

- **Password Hashing:** bcrypt ensures passwords are never stored in plain text.
- **JWT Expiry:** Tokens expire after a short duration (e.g., 1 hour) to reduce risk of misuse.
- **Refresh Tokens:** Longer-lived tokens used to issue new JWTs without requiring login again.
- **Two-Factor Authentication (2FA)** (future enhancement): Customers/mechanics may verify via SMS/email OTP.
- **Rate Limiting:** Prevents brute-force login attempts.
- **Audit Logs:** Records login attempts, failures, and suspicious activity.

### 5. Database Design for Auth

- **Users Table:**
  - user\_id (Primary Key)
  - email
  - phone\_number
  - password\_hash
  - role (customer/mechanic/admin)
  - status (active, suspended, pending verification)
- **Mechanics Table** (linked to Users):
  - mechanic\_id
  - verification\_status
  - documents\_uploaded
- **Tokens Table** (optional, for refresh tokens):
  - token\_id
  - user\_id
  - refresh\_token

**expiry\_time**

# Authorization (Access Control)

Authentication (Auth) verifies **who you are**, while **Authorization** determines **what you can do** once you're logged in. In Insta Mechanic, authorization is critical because customers, mechanics, and admins all have different responsibilities and permissions.

## 1. Purpose of Authorization

- **Role Separation:** Prevents unauthorized access to sensitive features.
- **Data Protection:** Ensures users only see data relevant to them.
- **System Integrity:** Stops mechanics from altering admin records or customers from bypassing payments.
- **Compliance:** Meets industry standards for secure access control.

## 2. Role-Based Access Control (RBAC)

Authorization in Insta Mechanic is implemented using **RBAC (Role-Based Access Control)**. Each role has a defined set of permissions:

- **Customer Role**
  - Can: Register/login, book services, make payments, track bookings, leave reviews.
  - Cannot: Access mechanic dashboards, override bookings, initiate refunds.
- **Mechanic Role**
  - Can: Accept jobs, update booking status (on\_the\_way, in\_progress, completed), view earnings.
  - Cannot: Access customer payment details, issue refunds, manage other mechanics.
- **Admin Role**
  - Can: Verify mechanics, monitor bookings, resolve disputes, initiate refunds, override statuses.
  - Cannot: Book services as a customer (unless using a customer account).

## 3. Middleware Enforcement

- **JWT Token Verification:**
  - Every request carries a JWT with the user's role.
  - Middleware checks the token and extracts the role.
- **Permission Check:**
  - Middleware compares the role against the requested route.
  - Example:
    - `/api/bookings/create` → Allowed for customers.
    - `/api/bookings/verify` → Allowed only for admins.
    - `/api/bookings/update` → Allowed for mechanics.
- **Access Denial:**
  - If role mismatch occurs, backend returns `403 Forbidden`.

#### 4. Database-Level Authorization

- **Row-Level Security (RLS)** in PostgreSQL:
  - Customers can only view their own bookings.
  - Mechanics can only view jobs assigned to them.
  - Admins can view all records.
- **Example Rule:**

sql

```
CREATE POLICY customer_policy ON bookings
FOR SELECT USING (user_id = current_user_id);
```

#### 5. Example Authorization Flow Diagram (Text-Based)

```
User → Login → JWT Issued (Role: Customer/Mechanic/Admin)
User → Request API Endpoint
Backend Middleware → Verify JWT → Extract Role
Middleware → Check Permissions
    → If Allowed → Execute Controller Logic
    → If Denied → Return 403 Forbidden
```

#### 6. Advanced Authorization Features

- **Granular Permissions:** Beyond roles, specific actions can be restricted (e.g., mechanics can update status but not delete bookings).
- **Attribute-Based Access Control (ABAC):** Future enhancement where rules depend on attributes like location, time, or booking status.
- **Audit Logging:** Every authorization decision is logged for compliance and debugging.
- **Temporary Privileges:** Admins can grant mechanics temporary access for special cases.

#### 7. Future Enhancements in Authorization

- **Multi-Tenant Support:** Different cities or regions managed separately with localized admins.
- **Dynamic Policy Updates:** Admins can change access rules without redeploying backend.
- **Zero Trust Model:** Continuous verification of every request, not just at login.
- **AI-Based Anomaly Detection:** Detect unusual access patterns (e.g., mechanic trying to access customer payment data).





# Conclusion

## *Summary of Findings*

- Insta Mechanic addresses the **core challenges** in the vehicle repair industry: lack of transparency, trust, and efficiency.
- The system design (client–server, booking lifecycle, payment lifecycle) ensures smooth operations.
- Deployment architecture demonstrates scalability and reliability.
- Security layers (JWT, bcrypt, SSL, audit logs) protect user data and transactions.
- Scalability strategies (load balancing, caching, async queues) prepare the system for high traffic.

## *Contributions of Insta Mechanic*

- **Academic Contribution:** Provides a comprehensive case study of applying modern web technologies to a real-world problem.
- **Industrial Contribution:** Offers a blueprint for digitizing mechanic services, similar to Uber’s impact on transportation.
- **Social Contribution:** Builds customer trust, empowers mechanics with digital tools, and ensures accountability through admin oversight.

## *Limitations of Current System*

### *1. Adoption Barriers*

One of the biggest challenges is **customer adoption**.

- **Trust Issues:** Many customers still prefer traditional garages due to long-standing relationships with local mechanics. Convincing them to shift to digital platforms requires building trust through transparency and consistent service quality.
- **Digital Literacy:** In rural and semi-urban areas, limited digital literacy can hinder adoption. Customers may struggle with app-based booking systems or online payments.
- **Resistance from Mechanics:** Mechanics accustomed to traditional practices may resist digital platforms, fearing loss of autonomy or increased monitoring. Training and incentives are necessary to overcome this barrier.

### *2. Cost and Infrastructure Challenges*

Implementing advanced technologies such as AI, blockchain, and IoT requires significant investment.

- **High Initial Costs:** Developing cloud-native microservices, integrating IoT sensors, and deploying blockchain infrastructure can be expensive.

- **Maintenance Costs:** Continuous updates, cybersecurity monitoring, and server maintenance add recurring expenses.
- **Infrastructure Gaps:** In rural areas, poor internet connectivity and lack of reliable logistics infrastructure limit scalability.

These challenges highlight the need for **cost-effective, hybrid models** that balance innovation with affordability.

### *3. Regulatory and Compliance Issues*

Digital platforms must navigate complex regulatory landscapes.

- **Data Privacy:** Compliance with GDPR, PCI DSS, and local data protection laws is mandatory. Mishandling customer data can lead to legal penalties and loss of trust.
- **Insurance and Liability:** Integrating insurance claim processing requires alignment with regulatory frameworks, which vary across regions.
- **Certification Standards:** Ensuring mechanics are certified and services meet safety standards is essential but difficult to enforce across decentralized networks.

Failure to address these issues can undermine credibility and hinder expansion.

### *4. Mechanic Training and Skill Gaps*

Mechanics form the backbone of these platforms, but skill gaps pose challenges.

- **Technology Familiarity:** Mechanics may lack exposure to digital tools, IoT diagnostics, or blockchain-based certification systems.
- **Continuous Learning:** With the rise of electric vehicles (EVs) and advanced automotive technologies, mechanics must undergo regular training.
- **Standardization:** Ensuring consistent service quality across diverse mechanics is difficult without standardized training modules.

Platforms must invest in **training programs, certifications, and digital literacy initiatives** to empower mechanics.

### *5. Cybersecurity Risks*

As platforms grow more complex, they become attractive targets for cyberattacks.

- **Data Breaches:** Customer and vehicle data are sensitive; breaches can damage reputation.
- **Fraudulent Activities:** Fake reviews, duplicate bookings, and inflated billing remain risks.
- **System Vulnerabilities:** Weak authentication or outdated software can expose platforms to attacks.

Implementing **Zero Trust security, AI-driven anomaly detection, and blockchain-based transparency** is essential but resource-intensive.

## *6. Ecosystem Integration Challenges*

While ecosystem integration is a key goal, it poses difficulties.

- **Coordination Across Services:** Integrating repair, towing, insurance, and preventive maintenance requires seamless coordination, which is complex to achieve.
- **Third-Party Dependencies:** Reliance on external partners (insurance companies, towing services) can create bottlenecks.
- **Customer Experience:** Ensuring a smooth, unified experience across multiple services is challenging.

Without effective integration, platforms risk becoming fragmented and losing customer loyalty.

## *7. Rural Accessibility*

Extending services to rural areas remains a major challenge.

- **Connectivity Issues:** Poor internet and mobile coverage limit digital adoption.
- **Logistics:** Delivering spare parts and deploying mechanics in remote areas is costly and inefficient.
- **Affordability:** Rural customers may be price-sensitive, requiring low-cost service models

Hybrid approaches combining **mobile mechanics, offline functionality, and local partnerships** are needed to overcome these barriers.

# Future Scope

The future of digital vehicle repair platforms lies in their ability to evolve into **autonomous, intelligent, and fully integrated ecosystems**. While current models have introduced convenience and transparency, the next generation will leverage **emerging technologies** such as **Generative AI, Digital Twins, 5G connectivity, and advanced cybersecurity frameworks** to redefine industry standards. This section explores the future scope in detail.

## *1. Generative AI for Personalized and Autonomous Services*

Generative AI will play a transformative role in enhancing customer experience and operational efficiency.

- **Personalized Recommendations:** AI models can analyze vehicle history, driving behavior, and environmental conditions to generate tailored maintenance schedules and service packages.
- **Automated Customer Support:** Chatbots powered by generative AI will provide instant troubleshooting, explain repair processes, and even simulate cost estimates in natural language.
- **Mechanic Training Simulations:** Generative AI can create realistic training environments, helping mechanics practice complex repairs virtually before applying them in real-world scenarios.
- **Dynamic Service Design:** AI can autonomously design new service workflows, optimizing resource allocation and reducing downtime.

This ensures that platforms move beyond static booking systems to **adaptive, customer-centric ecosystems**.

## *2. Digital Twins for Predictive Maintenance*

Digital twin technology will revolutionize vehicle diagnostics and preventive care.

- **Virtual Vehicle Models:** Each vehicle can have a digital twin that mirrors its physical state, continuously updated through IoT sensors.
- **Predictive Analytics:** By simulating wear and tear, digital twins can forecast potential failures, enabling proactive interventions.
- **Scenario Testing:** Mechanics can test repair strategies on the digital twin before applying them to the actual vehicle, reducing risks and errors.
- **Fleet Management:** For commercial operators, digital twins allow monitoring of entire fleets, optimizing maintenance schedules and reducing operational costs.

This technology shifts the industry from **reactive repairs to proactive, data-driven maintenance**.

## *3. 5G Connectivity for Real-Time Operations*

The rollout of 5G networks will enhance the speed, reliability, and scalability of digital platforms.

- **Ultra-Low Latency:** Real-time diagnostics and mechanic allocation will be possible, improving responsiveness.
- **Enhanced IoT Integration:** Vehicles equipped with IoT sensors can transmit large volumes of data instantly, supporting predictive maintenance and digital twin updates.
- **Remote Assistance:** Mechanics can guide customers through minor repairs via augmented reality (AR) applications powered by 5G.
- **Scalable Ecosystems:** 5G enables seamless integration of multiple services—repair, towing, insurance—into a unified platform without performance bottlenecks.

This connectivity ensures that platforms operate as **real-time, always-available ecosystems**.

#### *4. Advanced Cybersecurity and Zero Trust Models*

As platforms grow more complex, cybersecurity becomes critical.

- **Zero Trust Security:** Every user, device, and transaction is continuously verified, reducing risks of insider threats.
- **Blockchain Integration:** Tamper-proof records ensure transparency in billing, mechanic certifications, and insurance claims.
- **AI-Driven Threat Detection:** Machine learning models can detect anomalies in user behavior, preventing fraud and cyberattacks.
- **Privacy Compliance:** Adherence to GDPR and PCI DSS ensures customer data is protected, building trust and credibility.

Future platforms will not only deliver services but also guarantee **secure, transparent, and compliant operations**.

#### *5. Ecosystem Expansion and Integration*

The future scope extends beyond repair services to holistic mobility ecosystems.

- **Insurance Integration:** Platforms can automate claim processing, linking repair records directly to insurance providers.
- **Towing and Emergency Services:** Real-time GPS allocation ensures quick response during breakdowns.
- **Sustainable Practices:** Integration with electric vehicle (EV) charging networks and recycling services supports environmental goals.
- **Smart City Integration:** Platforms can collaborate with municipal systems for traffic management, accident response, and public safety.

This expansion positions platforms as **mobility hubs rather than isolated repair services**.

#### *6. Rural Accessibility and Inclusivity*

Future platforms must address the digital divide by extending services to rural areas.

- **Hybrid Models:** Combining mobile mechanics with local partnerships ensures coverage in underserved regions.
- **Offline Functionality:** Platforms can offer limited services without internet, syncing data once connectivity is restored.
- **Affordable Packages:** Subscription-based preventive maintenance tailored for rural customers ensures inclusivity.

This ensures that digital transformation benefits **both urban and rural communities**.

## *7. Sustainability and Green Technologies*

Environmental sustainability will be a key driver of innovation.

- **EV Integration:** Platforms will specialize in electric vehicle diagnostics, battery health monitoring, and charging infrastructure.
- **Carbon Footprint Tracking:** Customers can monitor the environmental impact of their vehicle usage and repairs.
- **Green Supply Chains:** Blockchain can ensure transparency in sourcing eco-friendly spare parts.

This aligns platforms with global sustainability goals and customer expectations.

## *Final Reflection*

Insta Mechanic is not just a booking platform—it is a **comprehensive ecosystem for vehicle care**. By combining modern web technologies, secure payment systems, and scalable architecture, it sets a foundation for future innovation in the automotive service industry.

# Appendices

The appendices provide **technical and supplementary details** that support the main chapters.

## *Appendix A: Database Schema*

- Tables: Users, Mechanics, Services, Bookings, Payments, Reviews, Vehicles.
- Example ER diagram with relationships.
- Sample SQL queries for booking creation and payment updates.

## *Appendix B: Code Snippets*

- Node.js controllers (AuthController, BookingController, PaymentController).
- Example of JWT authentication middleware.
- PostgreSQL query for mechanic availability.

## *Appendix C: System Screenshots*

- Customer booking interface.
- Mechanic dashboard.
- Admin panel for verification and refunds.

#### ***Appendix D: API Documentation***

- Endpoints for booking, payments, reviews, and authentication.
- Request/response formats (JSON).
- Error handling conventions.

#### ***Appendix E: Survey & Interview Data***

- Customer survey results on pain points.
- Mechanic interviews about operational challenges.
- Admin feedback on dispute resolution needs



# Acknowledgement

I would like to express my deepest gratitude to all those who contributed to the successful completion of this research work. First and foremost, I am thankful to my mentors and academic guide Mr. Rakesh Arya Sir whose valuable insights, constructive feedback, and continuous encouragement helped shape the direction of this study. Their expertise in technical writing, research methodology, and project management provided the foundation upon which this work was built.

I am also grateful to my peers and colleagues who offered thoughtful discussions and shared resources that enriched the quality of this research. Their support during late-night study sessions and collaborative brainstorming ensured that every section was refined with clarity and depth.

Special thanks are due to the developers and innovators behind existing platforms such as GoMechanic, YourMechanic, and UrbanClap, whose pioneering work inspired the exploration of gaps and opportunities in digital vehicle repair services. Their contributions to the industry provided the real-world context necessary for this study.

I acknowledge the role of technology providers and open-source communities whose tools, frameworks, and documentation made it possible to analyze system architectures, security models, and predictive maintenance technologies in detail. Their dedication to knowledge-sharing continues to empower researchers and practitioners alike.

Finally, I extend heartfelt appreciation to my family and close supporters for their patience, understanding, and encouragement throughout this journey. Their unwavering belief in my ability to complete this work gave me the strength to persevere through challenges and maintain focus on my academic goals.

This research is the result of collective effort, guidance, and inspiration from many sources, and I sincerely acknowledge each contribution with gratitude.

# References

Here is a structured set of references you can include in your research paper. I've organized them into **academic literature**, **industry reports**, and **technology standards/frameworks** to cover both scholarly and practical perspectives.

## Academic Literature

1. Chen, D., & Zhang, H. (2014). *Data-intensive applications, challenges, techniques and technologies: A survey on Big Data*. Information Sciences, 275, 314–347.
2. Ghosh, S., & Chatterjee, S. (2020). *Artificial Intelligence in predictive maintenance: A review*. Journal of Intelligent Manufacturing, 31(6), 1383–1399.
3. Lee, J., Bagheri, B., & Kao, H. A. (2015). *A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems*. Manufacturing Letters, 3, 18–23.
4. Wang, L., Törngren, M., & Onori, M. (2015). *Current status and advancement of cyber-physical systems in manufacturing*. Journal of Manufacturing Systems, 37, 517–527.
5. Porter, M. E. (2001). *Strategy and the Internet*. Harvard Business Review, 79(3), 62–78.

## Industry Reports & Case Studies

6. McKinsey & Company. (2018). *The future of mobility: Disruptive trends in automotive aftersales*.
7. Deloitte Insights. (2020). *Digital transformation in automotive services*.
8. PwC. (2019). *Connected car report: The digital transformation of the automotive industry*.
9. Frost & Sullivan. (2021). *Global automotive repair and maintenance services market outlook*.
10. KPMG. (2020). *Automotive industry trends: The road ahead*.

## Technology Standards & Frameworks

11. PCI Security Standards Council. (2018). *Payment Card Industry Data Security Standard (PCI DSS) Requirements and Security Assessment Procedures*.
12. European Union. (2016). *General Data Protection Regulation (GDPR)*.
13. National Institute of Standards and Technology (NIST). (2020). *Zero Trust Architecture*.
14. Kubernetes Documentation. (2022). *Production-grade container orchestration*.
15. IEEE Standards Association. (2019). *IEEE 802.11 Wireless LAN Standards*.

## About the Author

This project was completed by **Kumar Raj Ranjan**, a Computer Science and Engineering (CSE) student at **Dev Bhoomi Uttarakhand University**. His academic interests lie in software development, system architecture, and the application of emerging technologies such as artificial intelligence and blockchain in real-world service ecosystems. Kumar Raj Ranjan has demonstrated strong skills in technical writing, project execution, and collaborative teamwork, which were essential in bringing this research to completion.

He was supported by his classmates **Vihan Moudgil** and **Umesh Roy**, who contributed significantly to the project's development and analysis. Together, the team worked on designing the conceptual framework, identifying research gaps in existing platforms, and proposing innovative solutions for a secure, scalable, and customer-centric digital vehicle repair ecosystem.

The collaborative effort reflects not only their technical expertise but also their ability to work cohesively as a team, balancing academic rigor with practical innovation. Their shared vision was to create a platform that addresses the limitations of current models while aligning with global trends in predictive maintenance, fraud prevention, and ecosystem integration.

This work represents the team's commitment to advancing knowledge in the field of **digital transformation in automotive services**, and it stands as a testament to their dedication, creativity, and academic excellence.

### NAME OF THE AUTHOR:

**KUMAR RAJ RANJAN**

**VIHAN MOUDGIL**

**UMESH ROY**